VIETNAM NATIONAL UNIVERSITY, HANOI
VIETNAM JAPAN UNIVERSITY

NGUYEN HUY DAT

SLAM-BASED LOCALIZATION AND MAPPING IN HAZARDOUS AND INACCESSIBLE ENVIRONMENTS

Undergraduate thesis
Major: Computer Science and Engineering
Major code: 7480204QTD
Undergraduate program in: Computer Science and Engineering

Hanoi - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
VIETNAM JAPAN UNIVERSITY

NGUYEN HUY DAT

SLAM-BASED LOCALIZATION AND MAPPING IN HAZARDOUS AND INACCESSIBLE ENVIRONMENTS

Undergraduate thesis
Major: Computer Science and Engineering
Major code: 7480204QTD
Undergraduate program in: Computer Science and Engineering

SUPERVISOR: Dr. Duong Huu Toan

Hanoi - 2026

DECLARATION

This thesis/graduation project is the result of my study and training at Vietnam Japan University.

Under the supervision of **Dr. Duong Huu Toan**, I have carried out this research with seriousness and dedication.

The findings and content presented in this thesis/project truthfully reflect my own work and have not been published elsewhere.

All references and data sources are properly cited in accordance with regulations.

I take full responsibility for the integrity and authenticity of the entire thesis/project.

Hanoi, date….month 08 year 2026.

Student

Nguyen Huy Dat

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude and sincere appreciation to my academic supervisor, **Dr. Duong Huu Toan**, for his invaluable guidance, continuous encouragement, and profound technical insights throughout the realization of this undergraduate thesis. His persistent feedback during weekly reporting sessions greatly refined my understanding of robotics system integration, state estimation, and sensor fusion algorithms.

I am also profoundly grateful to the faculty members and staff of the Computer Science and Engineering Program at Vietnam Japan University (VJU - VNU) for providing a rich academic environment, state-of-the-art laboratory facilities, and fundamental knowledge that formed the bedrock of my engineering capabilities.

Lastly, I extend my heartfelt thanks to my family and friends for their unyielding moral support, patience, and motivation during long hours of hardware debugging, circuit soldering, and algorithm refinement.

I am deeply grateful to you.

LIST OF ABBREVIATIONS

Abbreviation	Full Meaning
2WD	Two-Wheel Differential Drive
APE	Absolute Pose Error
CPU	Central Processing Unit
DC	Direct Current
DOF	Degrees of Freedom
EKF	Extended Kalman Filter
ESP32	Espressif Systems 32-bit Microcontroller
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
M3RSM	Multi-Resolution Multi-Threaded Correlative Scan Matching
MEMS	Micro-Electro-Mechanical Systems
PGO	Pose Graph Optimization
PWM	Pulse Width Modulation
ROS2	Robot Operating System 2
RPE	Relative Pose Error

Abbreviation	Full Meaning
RPM	Revolutions Per Minute
SLAM	Simultaneous Localization and Mapping
TF	Transform Framework
UART	Universal Asynchronous Receiver-Transmitter
UGV	Unmanned Ground Vehicle
URDF	Unified Robot Description Format

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

Simultaneous Localization and Mapping (SLAM) is a foundational capability for autonomous mobile robots operating in GPS-denied, hazard-prone, and confined spaces such as narrow caves, underground drainage tunnels, and collapsed infrastructure. In these harsh environments, standalone LiDAR-based SLAM algorithms frequently fail due to geometric degeneracy in long featureless corridors and wheel slip over uneven or slippery terrain. This thesis presents a robust, low-cost Two-Wheel Differential Drive Unmanned Ground Vehicle (2WD UGV) system leveraging Sensor Fusion through a Generalized Extended Kalman Filter (EKF) and SLAM Toolbox to maintain highly accurate 2D mapping and localization.

The primary contribution of this research lies in the development of a multi-threaded ROS2 framework integrated with an ESP32 micro-controller bridge. A low-level firmware architecture handles high-frequency quadrature encoder decoding (50Hz) and MPU-6050 IMU calibration with deadzone filtering. To combat physical terrain hazards, a novel Dual-Threshold Safety Reflex (Bump Reflex) and Dynamic Covariance mechanism is implemented within the C++ controller node. By monitoring longitudinal acceleration (A_x) and angular mismatch between encoder kinematics and IMU gyroscope measurements, the system detects collisions, wall scrapes, and wheel slip in real-time. Upon fault detection, the controller executes emergency braking and dynamically inflates the wheel odometry covariance matrix ($\sigma^2 = 100.0$), forcing the EKF to reject corrupt wheel odometry and rely strictly on inertial measurements.

Experimental validation was conducted in a real-world testing corridor across two comparative configurations: Case 1 (LiDAR + Encoder) and Case 2 (LiDAR + EKF Fusion with Encoder and IMU). Quantitative results demonstrate that the EKF sensor fusion architecture reduces corridor translational distance error from **19.54% down to 7.81%** over a 52.5m

featureless path, eliminates catastrophic rotational collapse during in-place pivot turns, and maintains structural map topology despite a **3.18m** post-loop closure trajectory drift caused by corridor blindness. The integrated platform provides a highly reliable, fault-tolerant solution for mapping hazardous environments at a total hardware cost under $50.

Keywords: Autonomous Mobile Robots, SLAM Toolbox, Extended Kalman Filter, Sensor Fusion, Wheel Slip Detection, ROS2, Hazardous Environments.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1. Context and Research Motivation

In recent years, the deployment of autonomous Unmanned Ground Vehicles (UGVs) in extreme, hazardous, and human-inaccessible environments has become a vital area of research within field robotics. Environments such as narrow natural caves, underground drainage pipes, collapsed mine shafts, and post-disaster structural ruins pose severe threats to human safety due to the presence of toxic natural gases, structural instabilities, complete darkness, and severe spatial confinement. In these critical scenarios, sending human responders or search teams for structural inspection, eco-system monitoring, or search-and-rescue operations is highly dangerous or physically impossible.

To operate effectively in these challenging domains, a mobile robot must possess the capability to navigate autonomously, locate itself in real time, and reconstruct a precise two-dimensional (2D) or three-dimensional (3D) spatial representation of its surroundings without relying on external positioning infrastructure. This fundamental capability is known as Simultaneous Localization and Mapping (SLAM) (Thrun et al., 2005). While satellite-based positioning systems (such as GPS or GNSS) serve as the primary localization backbone for outdoor autonomous systems, electromagnetic signals cannot penetrate dense soil, subterranean rock formations, or concrete walls, rendering subterranean and indoor environments completely GPS-denied.

Consequently, mobile UGVs operating in confined subterranean spaces must rely entirely on onboard perception sensors - such as Light Detection and Ranging (LiDAR), Inertial Measurement Units (IMUs), and wheel quadrature encoders - to infer their ego-motion and construct accurate environmental maps. However, executing SLAM in narrow, featureless corridors and rough subterranean terrain presents severe algorithmic and physical limitations. Pure LiDAR-based SLAM algorithms frequently suffer from geometric degeneracy

("corridor blindness") when traversing long, featureless walls where consecutive laser scans appear identical. Furthermore, rough, wet, or muddy subterranean surfaces induce unpredictable wheel slippage and loss of traction, causing wheel odometry calculations to severely drift and deform the generated occupancy map.

Motivated by these operational challenges, this thesis focuses on engineering a cost-effective, highly reliable, and fault-tolerant mobile UGV system capable of high-precision SLAM in hazardous and confined environments by utilizing sensor fusion techniques and robust control strategies within the Robot Operating System 2 (ROS2) framework (Quigley et al., 2009).

1.2. Problem Statement

While modern 2D Graph-Based SLAM algorithms, such as ROS2 SLAM Toolbox, demonstrate high performance in structured office environments, their mapping integrity rapidly degrades when deployed on low-cost differential drive platforms operating on non-ideal terrain. The core technical problems addressed in this research stem from three primary physical and algorithmic failure modes:

First, **Wheel Odometry Drift and Slip Accumulation:** Differential drive UGVs estimate their incremental movement by counting motor encoder pulses. On rough, slippery, or uneven surfaces, wheels experience mechanical slip (spinning without forward motion) or momentary air-spinning when encountering small obstacles. Traditional dead-reckoning equations process these false rotation counts as actual vehicle displacement, injecting massive position and heading errors into the local coordinate frame. Over time, these cumulative errors distort the topological graph of the SLAM solver, leading to ghosting artifacts, double-wall phenomena, and overlapping maps.

Second, **Geometric Degeneracy in Featureless Corridors:** In narrow underground tunnels or smooth corridors, laser beams emitted by 2D LiDAR sensors strike parallel, featureless walls. Because scan matching algorithms (Grisetti et al., 2007) (such as Iterative Closest Point or Correlative Scan Matching) rely on structural geometric features (corners, pillars, edges) to

resolve relative displacement, traversing a featureless corridor leaves the scan matcher mathematically constrained along the lateral axis but completely unconstrained along the longitudinal axis. As a result, the LiDAR scan matcher becomes "blind" along the corridor direction, failing to detect forward motion accurately without reliable odometric assistance.

Third, **Lack of Physical Fault Detection and Sensor Rejection:** Standard ROS2 navigation pipelines assume that incoming sensor data streams are continuously valid within static noise bounds. When a low-cost UGV collides with an unmapped obstacle or scrapes against a tunnel wall, the motor drive continues to command wheel rotation, generating corrupt odometry velocity spikes. Existing SLAM frameworks lack an embedded safety reflex to dynamically isolate and reject corrupted wheel odometry during physical collision or heavy slip events.

1.3. Research Objectives

To overcome the aforementioned limitations, this thesis aims to design, construct, and evaluate a low-cost, fault-tolerant 2WD UGV platform integrated with multi-sensor fusion algorithms to achieve robust 2D mapping and state estimation in hazardous, GPS-denied environments. The specific research objectives are as follows:

1. Hardware Platform and Microcontroller Architecture Integration: Design and assemble a lightweight, low-cost Two-Wheel Differential Drive UGV hardware platform equipped with an ESP32 micro-controller bridge, low-RPM high-torque DC encoder motors (JGA25-370), an MPU-6050 6-DOF IMU, a 2D LiDAR (LD14), and a dedicated motor driver (TB6612FNG) powered by a regulated 11.1V lithium-ion battery pack.

2. High-Frequency Embedded Firmware Development: Develop an optimized ESP32 embedded C++ firmware utilizing hardware interrupts for dual quadrature encoder pulse counting (50Hz), static sensor calibration, deadzone

filtering for IMU gyroscope drift, and high-speed bidirectional UART communication with the main host computer.

3. Generalized EKF Sensor Fusion Implementation: Implement a multi-sensor Generalized Extended Kalman Filter (EKF) state estimation node within ROS2. Construct a kinematically decoupled prediction model where state propagation relies on differential motion equations, while update phases dynamically fuse longitudinal velocity (V_x) from encoders and yaw rate (ω_z) from the MPU-6050 gyroscope to eliminate heading drift and wheel slip distortions.

4. Dual-Threshold Safety Reflex and Dynamic Covariance Mechanism: Design and implement a novel fault-detection C++ controller node (`base_controller_node`) utilizing a dual-threshold acceleration trigger (A_x) and angular velocity mismatch monitor ($|v_{yaw\_wheel} - \omega_{imu}|$). When physical collisions, wall scrapes, or wheel slippage occur, the system automatically executes emergency braking and dynamically inflates wheel odometry covariance ($\sigma^2 = 100.0$), forcing the EKF to reject corrupt wheel data.

5. Quantitative System Evaluation and Comparative Analysis: Conduct rigorous physical experiments in a narrow hallway environment to compare two distinct system configurations: Case 1 (LiDAR + Encoder) versus Case 2 (LiDAR + EKF Fusion with Encoder and IMU). Evaluate translational distance accuracy, rotational fidelity, absolute trajectory drift (d), map wall thickness, and closed-loop closure integrity using evaluation tools.

1.4. Scope of the Thesis

The scope of this undergraduate thesis is delineated by the following technical boundaries:

• **Spatial Domain:** The research focuses strictly on 2D planar mapping and localization (3 Degrees of Freedom: x, y, and yaw angle θ). While 3D pitch

and roll acceleration components from the IMU are monitored for collision detection, full 3D volumetric SLAM is outside the current scope.

• **Perception Hardware:** Primary spatial sensing is restricted to a 360-degree 2D triangulation LiDAR (LD14) operating at 10Hz with a maximum effective range of 8 meters, complemented by a 6-DOF MEMS IMU (MPU-6050) and dual magnetic motor encoders.

• **Software Framework:** Software development is conducted within the Ubuntu 22.04 LTS environment using ROS2 Humble Hawksbill. Local and global mapping pipelines rely on ROS2 SLAM Toolbox (Karto SLAM core optimized with Ceres Solver) and `robot_localization` EKF packages.

1.5. Key Contributions of the Work

The primary technical contributions of this thesis are summarized below:

1. End-to-End Cost-Effective Robotic System Design: Successfully built a functional, highly accurate autonomous SLAM platform with a total component hardware cost under $50, establishing a practical benchmark for low-cost research robotics.

2. Novel Dual-Threshold Safety Reflex (Bump Reflex) Algorithm: Developed an embedded fault-detection logic within the C++ velocity controller node that successfully categorizes direct collisions ($|A_x| > 4.0 m/s^2$), wall scraping ($|A_x| > 2.0 m/s^2$), and soft mud slippage ($yaw\_mismatch > 0.5 rad/s$), triggering real-time motor inhibition.

3. Dynamic Covariance Fusion Strategy: Formulated a dynamic sensor weighting mechanism that bridges low-level physical fault detection with high-level Bayesian state estimation, preventing corrupt odometric data from corrupting the EKF pose graph during slip events.

4. Rigorous Experimental Validation: Provided comprehensive quantitative proof that fusing IMU angular velocity with encoder wheel odometry via EKF reduces short-range distance errors from 7.59% down to

0.86% (over 5.8m) and long-range corridor distance errors from 19.54% down to 7.81% (over 52.5m), while eliminating mapping failures during in-place pivot turns in narrow corridors.

1.6. Organization of the Thesis

The remainder of this thesis is structured into four subsequent chapters as follows:

• **Chapter 2: System Architecture and Hardware Integration** details the physical robot mechanical construction, electrical circuit topology, sensor selection rationale, ESP32 embedded firmware logic, and ROS2 static coordinate transformation tree (TF Tree).

• **Chapter 3: Algorithm Design and Sensor Fusion Framework** presents the mathematical formulations of the differential drive kinematic model, the Generalized EKF prediction/update steps, the Dual-Threshold Bump Reflex logic with Dynamic Covariance, and the SLAM Toolbox pose graph optimization pipeline.

• **Chapter 4: Experimental Results and Performance Evaluation** presents empirical testing methodologies, quantitative trajectory measurements, comparative mapping results between Case 1 and Case 2, and verification of the safety reflex mechanism.

• **Chapter 5: Conclusion and Future Work** summarizes the key achievements of the project, outlines current hardware/software limitations, and proposes directions for future research expansion.

CHAPTER 2: SYSTEM ARCHITECTURE AND HARDWARE INTEGRATION

2.1. Overall Mechanical Construction and Platform Layout

The physical structure of the autonomous Unmanned Ground Vehicle (UGV) developed in this research is engineered around a Two-Wheel Differential Drive (2WD) mechanical kinematics model. The chassis is constructed from a single lightweight transparent acrylic base plate designed specifically for high maneuverability within tight, confined subterranean passages, drainage culverts, and narrow cave openings. Differential drive mobile platforms offer a fundamental mechanical advantage in restricted spatial environments: by independently driving the left and right wheels at identical speeds in opposing directions, the vehicle achieves zero-radius in-place turning (pivot rotation). This zero-turning-radius capability allows the robot to reverse direction or maneuver around tight obstacles in corridors that are barely wider than the physical width of the chassis itself.

The mechanical layout comprises two primary active drive wheels mounted symmetrically along a central transverse axis ($L = 0.194m$ wheel base), coupled with a passive omnidirectional caster wheel positioned at the rear of the single acrylic base plate to maintain three-point planar stability. The active drive wheels feature high-traction rubber tires with an effective radius of $r = 0.034m$ (68mm outer diameter), providing adequate ground clearance and friction coefficient on smooth indoor flooring and slightly uneven subterranean surfaces.

Component spatial arrangement was meticulously planned to optimize weight distribution, maintain a low center of gravity, and preserve coordinate frame symmetry. The bottom surface of the single acrylic plate mounts the DC gear motors, wheel assemblies, and quadrature encoders. The upper surface of the acrylic chassis accommodates the main electronics prototyping board

(containing the ESP32 microcontroller, TB6612FNG driver, and MPU-6050 IMU) alongside the battery holder. The heavy 3-cell 18650 lithium-ion battery pack is positioned at the rear on the upper surface of the acrylic plate - located directly above the rear caster wheel mounted underneath - to balance the forward motor mass and maintain consistent tire ground contact. The primary sensing payload, the 2D LiDAR LD14, is screwed directly on top of the MPU-6050 IMU on the central electronic board along the longitudinal axis, creating a compact vertical sensor stack that eliminates extra structural decks while guaranteeing an unobstructed 360° horizontal laser scanning plane. The overall bill of materials for the hardware platform totals approximately $50, demonstrating exceptional cost efficiency for research-grade field robotics.

Figure 2.1: Mechanical Baseplate and Hardware Stack Layout

2.2. Electrical Topology and Hardware Component Selection

The electrical system architecture is designed to establish high-speed data acquisition, noise-immune sensor interconnections, and robust power regulation. The hardware components were selected based on strict criteria regarding power consumption, form factor, sampling rate, and ROS2 compatibility.

1. ESP32 Microcontroller Bridge: The core low-level processing unit is the Espressif Systems ESP32 microcontroller, featuring a dual-core 32-bit Xtensa LX6 processor operating at 240MHz. The ESP32 acts as a hardware bridge between low-level sensor electronics and the high-level ROS2 host computer (laptop). It executes real-time hardware interrupt processing for quadrature encoders, reads MPU-6050 IMU register data over I2C, computes basic motor control outputs, and streams formatted sensor telemetry to the host PC via a high-speed USB-to-UART serial interface at 115,200 baud.

2. DC Motors with Integrated Magnetic Encoders: Propulsion is driven by two JGA25-370 high-torque DC gear motors operating at a nominal voltage of 12V with a no-load speed rating of 130 RPM (Revolutions Per Minute). Selecting a lower 130 RPM gear ratio ensures high torque output required for traversing rough subterranean terrain while enforcing a controlled, slow velocity profile ($\sim 0.2 - 0.3 m/s$). Slower movement significantly improves LiDAR point cloud density and encoder pulse counting precision. Each motor incorporates a dual-channel Hall effect magnetic quadrature encoder attached to the rear armature shaft, delivering 11 Pulses Per Revolution (PPR) at the motor shaft. Factoring in the internal 1:34 gear reduction ratio, the encoder yields an effective resolution of 494.5 ticks per wheel revolution ($N_{ticks} = 494.5$), translating to a distance resolution of $0.432mm$ per pulse tick ($METERS\_PER\_TICK = 0.000432m$).

3. TB6612FNG Dual Motor Driver: Motor power switching is controlled by the Toshiba TB6612FNG MOSFET-based dual H-bridge motor driver module. Compared to legacy bipolar L298N drivers, the TB6612FNG

exhibits significantly lower internal resistance, negligible thermal dissipation, and higher operational efficiency, supplying up to 1.2A continuous current per channel (3.2A peak). PWM (Pulse Width Modulation) speed control signals are generated by the ESP32 hardware LEDC peripheral across a 0 to 255 duty cycle spectrum.

4. LiDAR LD14 (2D Laser Scanner): Spatial perception is provided by the LDROBOT LD14 360-degree 2D triangulation LiDAR scanner. Operating at a scanning frequency of 10Hz, the LD14 emits infrared laser pulses across a 360-degree horizontal plane, measuring distance via time-of-flight triangulation across a range from 0.15m to 8.0m. To prevent UART buffer overflow and CPU bottlenecking on the ESP32 microcontroller during high-speed point cloud transmission, the LD14 data line bypasses the ESP32 entirely and connects directly to the host PC via a dedicated CP2102 USB-to-UART bridge module.

5. MPU-6050 6-DOF IMU Sensor: Inertial sensing is captured by the InvenSense MPU-6050 MEMS sensor (InvenSense Inc., 2013), combining a 3-axis accelerometer and a 3-axis gyroscope on a single silicon die. Communication with the ESP32 is established over the I2C bus at a 400kHz clock frequency. The accelerometer is configured to a full-scale range of $\pm 2g$, while the gyroscope operates at $\pm 250^{\circ}/s$ with a Digital Low-Pass Filter (DLPF) bandwidth set to 21Hz to suppress high-frequency mechanical motor vibrations.

6. Regulated Power Supply Architecture: The power distribution subsystem is energized by a 3-cell 18650 Lithium-Ion battery pack supplying a nominal voltage of $\sim 11.1V$ ($\sim 2500mAh/9250mWh$ per cell). Motor armatures receive raw battery voltage directly through the TB6612FNG driver to maximize torque. To protect sensitive digital logic from voltage sags and inductive kickback generated by motor switching, a buck step-down converter module (MP1584 fixed 5V output) steps the battery voltage down to a stable 5.0V supply dedicated exclusively to powering the ESP32 microcontroller and onboard sensors. The fixed 5V buck module eliminates variable potentiometers

that could shift under heavy physical vehicle vibration, ensuring complete circuit safety.

Table 2.1 summarizes the complete hardware technical specifications of the 2WD UGV platform.

Table 2.1: Hardware Component Specifications of the 2WD UGV Platform

Component	**Model / Specification**	**Key Performance Parameters**
Microcontroller	Espressif ESP32 NodeMCU	Dual-core 32-bit Xtensa @ 240MHz, Hardware Interrupts, I2C, UART 115200 baud
DC Gear Motors	2x JGA25-370 (12V DC)	130 RPM, Gear Ratio 1:34, High Torque for slow-speed subterranean navigation
Quadrature Encoders	Dual-Channel Magnetic Hall Effect	11 PPR motor shaft (494.5 PPR wheel output), $0.432mm/tick$ linear resolution
Motor Driver	Toshiba TB6612FNG Dual H-Bridge	MOSFET efficiency, 1.2A continuous / 3.2A peak per channel, 0-255 PWM control
2D LiDAR Sensor	LDROBOT LD14	360° Triangulation Laser, 10Hz scan rate, 0.15m - 8.0m range, CP2102 USB Bridge
IMU Sensor	InvenSense MPU-	6-DOF (3-axis Accel $\pm 2g$, 3-axis

Component	Model / Specification	Key Performance Parameters
	6050 MEMS	Gyro $\pm 250°/s$), I2C @ 400kHz, DLPF 21Hz
Voltage Regulator	MP1584 Fixed 5V Buck Module	Fixed 5V output, vibration-proof, isolates ESP32/IMU from motor inductive sags
Battery Source	3x 18650 Li-ion Cells in Series	Nominal 11.1V, $2500mAh/9250mWh$, dedicated motor drive & buck input

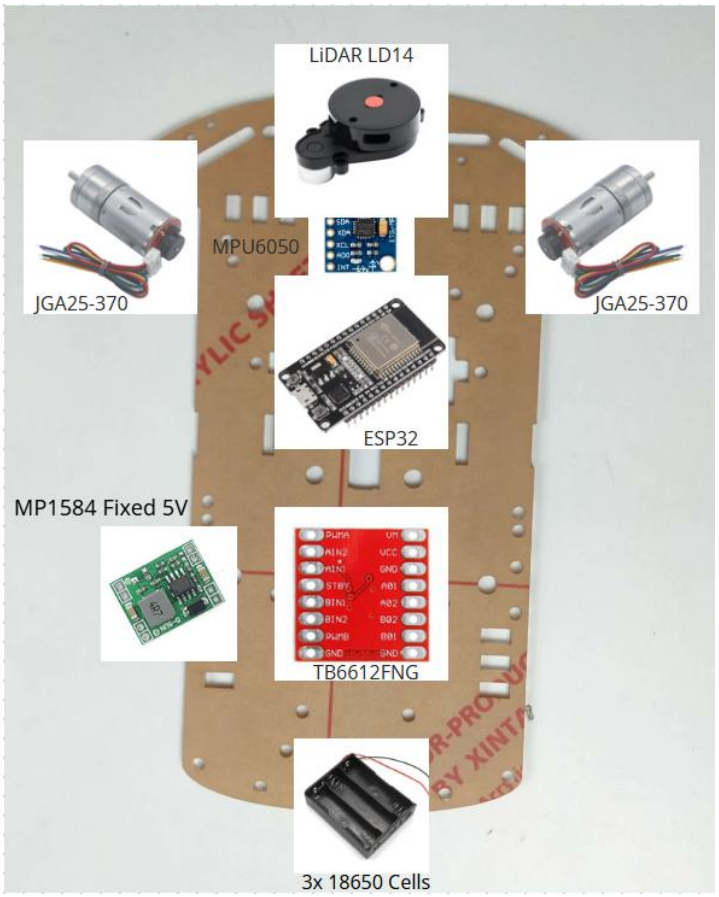


Figure 2.2: Overall System Architecture Diagram

The complete physical wiring interconnects, power distribution paths, and GPIO pin mapping across the ESP32 microcontroller, TB6612FNG motor driver, MPU-6050 IMU, MP1584 buck converter, and motor quadrature encoders are systematically illustrated in Figure 2.3.

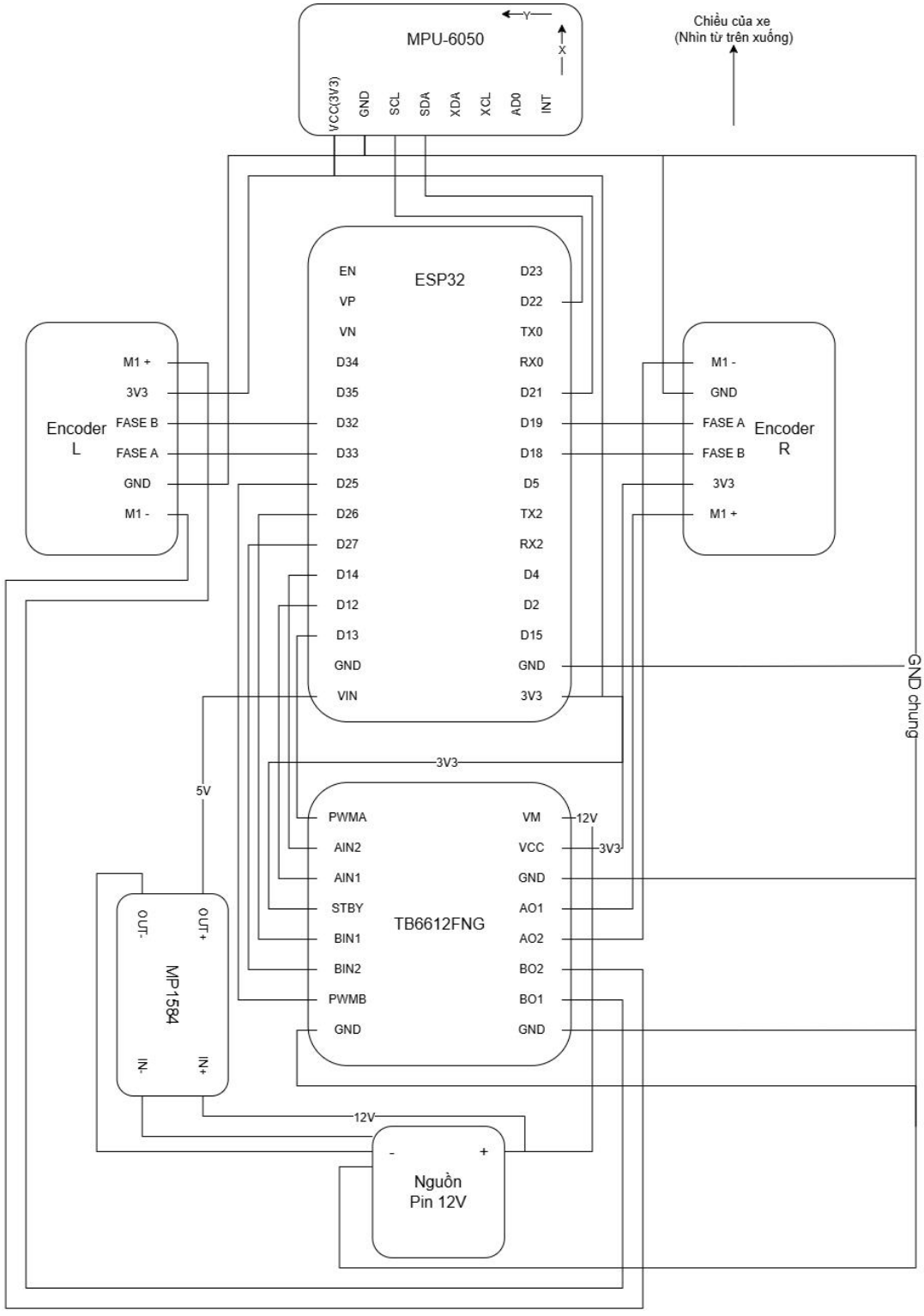


Figure 2.3: Detailed Electrical Circuit Topology and Hardware Connection Schematics

2.3. Embedded Firmware Architecture on ESP32 Microcontroller

The low-level firmware executed on the ESP32 microcontroller (`esp_robot_core.ino`) is structured around a real-time event loop designed to handle high-frequency sensor sampling and reliable motor actuation without blocking execution threads.

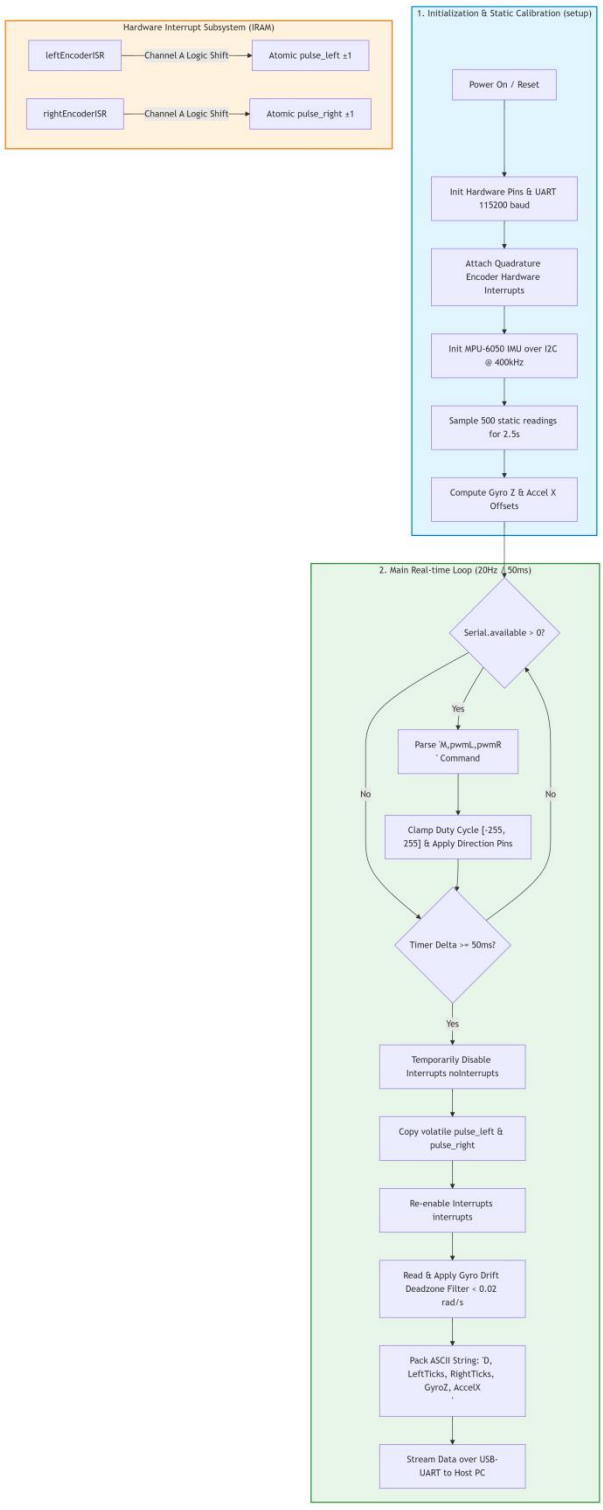


Figure 2.4: ESP32 Firmware Execution Flow and Interrupt Subsystem

1. Dual Quadrature Encoder Interrupt Subsystem: Accurate dead-reckoning requires capturing every single pulse state change generated by the motor hall sensors. The ESP32 firmware configures four dedicated GPIO pins (`ENC_L_A` Pin 32, `ENC_L_B` Pin 33, `ENC_R_A` Pin 19, `ENC_R_B` Pin 18) with internal pull-up resistors (`INPUT_PULLUP`). Hardware interrupts are attached to channel A pins using the `attachInterrupt(..., CHANGE)` function, mapping to Interrupt Service Routines (`leftEncoderISR` and `rightEncoderISR`) placed in IRAM memory (`IRAM_ATTR`) for microsecond-level execution speed. The ISR decodes directionality by comparing Channel A and Channel B logic levels, incrementing or decrementing atomic pulse counters (`pulse_left` and `pulse_right`).

2. MPU-6050 Calibration and Sensor Filtering: Upon power-up, the firmware executes a static calibration routine within `setup()`. The robot must remain stationary for 2.5 seconds while the ESP32 samples 500 consecutive readings from the MPU-6050 gyroscope and accelerometer registers over I2C. The routine computes static bias offsets for the Z-axis gyroscope (`gyro_z_offset`) and X-axis accelerometer (`accel_x_offset`):

$$gyro\_z\_offset = \frac{1}{500}\sum_{i=1}^{500} \omega_{z,i}^{raw}, \quad accel\_x\_offset = \frac{1}{500}\sum_{i=1}^{500} A_{x,i}^{raw}$$

During runtime, these offset constants are subtracted from raw sensor measurements. To eliminate static angular trift (gyroscope drift when the vehicle is stationary), an empirical deadzone filter is applied to the calibrated angular velocity: if $|\omega_z| < 0.02 rad/s$, the measurement is forced to $0.0 rad/s$.

3. Bidirectional Serial Protocol and Data Telemetry: Telemetry exchange between the ESP32 and the host PC occurs over USB Serial at 115,200 baud. To prevent serial buffer blocking from stalling the main execution loop, `Serial.setTimeout(10)` is enforced. At a deterministic frequency of 20Hz (50ms interval), the ESP32 packs sensor telemetry into a comma-separated ASCII string structured as follows:

$$Format: "D, \langle LeftTicks \rangle, \langle RightTicks \rangle, \langle GyroZ \rangle, \langle AccelX \rangle \setminus n"$$

When reading encoder pulse values, global interrupts are temporarily suspended (`noInterrupts()`) while copying volatile tick counters to local variables, preventing race conditions during atomic 32-bit register reads. Conversely, incoming motor velocity commands from the host PC are parsed from string commands formatted as `"M,pwmL,pwmR\n"`. The firmware constrains PWM duty cycle inputs strictly within $[-255,255]$, applies hardware motor directional pin logic (`AIN1`, `AIN2`, `BIN1`, `BIN2`), and executes an explicit software channel inversion (`pwmL = -pwmL`) to harmonize physical motor wiring with ROS2 forward velocity conventions.

2.4. Coordinate Transformation Framework (TF Tree) and Calibration

In ROS2, spatial relationships between robot chassis, sensors, and global reference frames are managed hierarchically by the Transform Framework (TF2). Every sensor data packet published into the ROS ecosystem must be associated with a specific coordinate frame (`frame_id`). The ROS2 node `robot_state_publisher` continuously broadcasts transformation matrices describing spatial translation (x, y, z) and rotation $(roll, pitch, yaw)$ across the coordinate tree.

1. Robot Base Frame (`base_link`): According to standard ROS REP-105 guidelines for 2WD differential drive robots, the root vehicle coordinate frame - designated as `base_link` - is defined as the geometric center point located on the transverse axis connecting the centers of the two active drive wheels, projected down to the ground plane surface ($z = 0$).

2. Static Transform Tree Configuration (`static_transform_publisher`): The physical offsets between `base_link` and perception sensors were measured precisely using vernier calipers and specified within the ROS2 launch file (`run_system_w_slamtoolbox.launch.py`) via `static_transform_publisher` nodes:

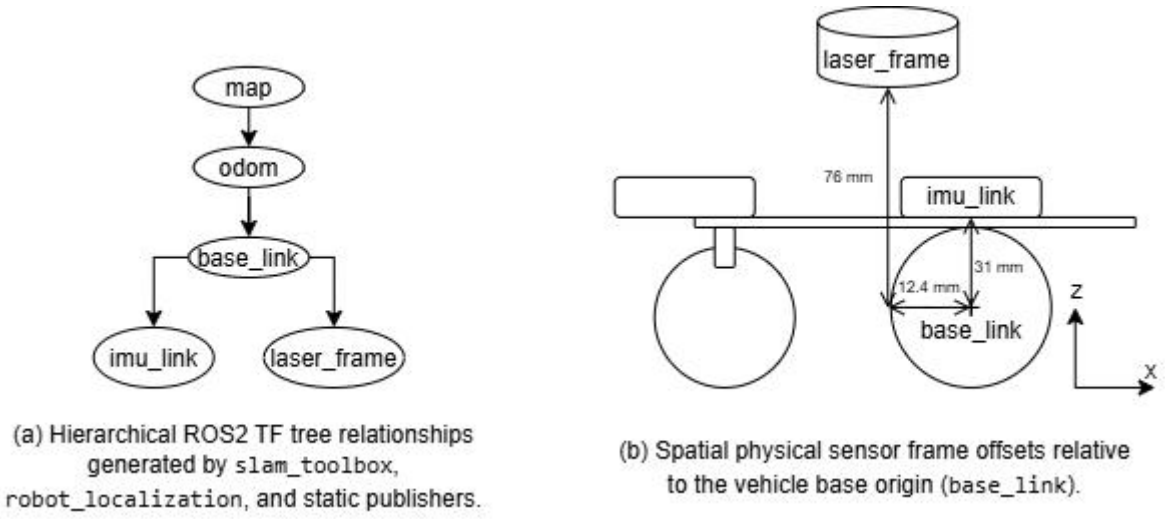
• **IMU Frame (`imu_link`):** The MPU-6050 sensor is mounted directly above `base_link` at a vertical height of 31mm ($z = +0.031m$) with zero spatial translation along X and Y axes and zero rotational offset:

$$\mathbf{T}_{base\_link}^{imu\_link} = [x: 0.0m, y: 0.0m, z: 0.031m, roll: 0°, pitch: 0°, yaw: 0°]$$

• **LiDAR Frame (`laser_frame`):** The LD14 laser scanner center is positioned on the upper deck, shifted slightly backward along the X-axis by 12.4mm ($x = -0.0124m$) due to mounting standoff alignment, and elevated at a total height of 76mm ($z = +0.076m$) above ground level:

$$\mathbf{T}_{base\_link}^{laser\_frame} = [x: -0.0124m, y: 0.0m, z: 0.076m, roll: 0°, pitch: 0°, yaw: 0°]$$

The spatial hierarchy of the coordinate transformation framework, alongside the physical extrinsic offsets connecting base_link to onboard sensing payloads (imu_link and laser_frame), is schematically illustrated in Figure 2.5.



(a) Hierarchical ROS2 TF tree relationships generated by `slam_toolbox`, `robot_localization`, and static publishers.

(b) Spatial physical sensor frame offsets relative to the vehicle base origin (`base_link`).

Figure 2.5: Coordinate Transformation Framework (TF Tree) of the 2WD UGV Platform

3. Extrinsic Calibration Rationale: Industrial robotics often employ automatic extrinsic sensor calibration algorithms (such as `kalibr` or `lidar_align`) to estimate transformation matrices non-linearly. However, manual physical caliper measurement ($\pm 5mm$ error bound) was selected for this 2WD UGV platform for two critical reasons:

First, **Grid Resolution Tolerance:** The configured spatial resolution of the SLAM Toolbox occupancy grid map is set to $0.05m$ ($50mm$ per grid pixel). The $\pm 5mm$ physical measurement tolerance represents only $1/10th$ of a single map cell, making its effect mathematically negligible and fully absorbed by the scan matching cost solver.

Second, **Planar Motion Observability Limits:** Automatic 3D extrinsic calibration algorithms require exercising the sensor payload through full 6-DOF rotational motion (rolling, pitching, vertical translation) along dynamic Figure-8 trajectories. Because 2WD differential drive vehicles operate strictly on a 2D planar surface (constrained to x, y, yaw), executing 6-DOF excitation motion is mechanically impossible, causing automatic calibration routines to report mathematical non-observability errors.

By establishing precise static TF transforms, ROS2 automatically executes homogeneous matrix multiplications to project 2D laser scan points from `laser_frame` directly into `base_link` and `odom` coordinate frames, ensuring highly accurate mapping input for SLAM Toolbox.

CHAPTER 3: ALGORITHM DESIGN AND SENSOR FUSION FRAMEWORK

This chapter details the mathematical formulations and algorithmic architecture governing the autonomous 2WD UGV. To provide a comprehensive overview of the complete processing pipeline, the proposed software stack is structured into a four-tier architecture - spanning low-level sensor telemetry acquisition, embedded safety reflex detection, Generalized EKF state estimation, and high-level pose graph SLAM - as systematically depicted in Figure 3.1.

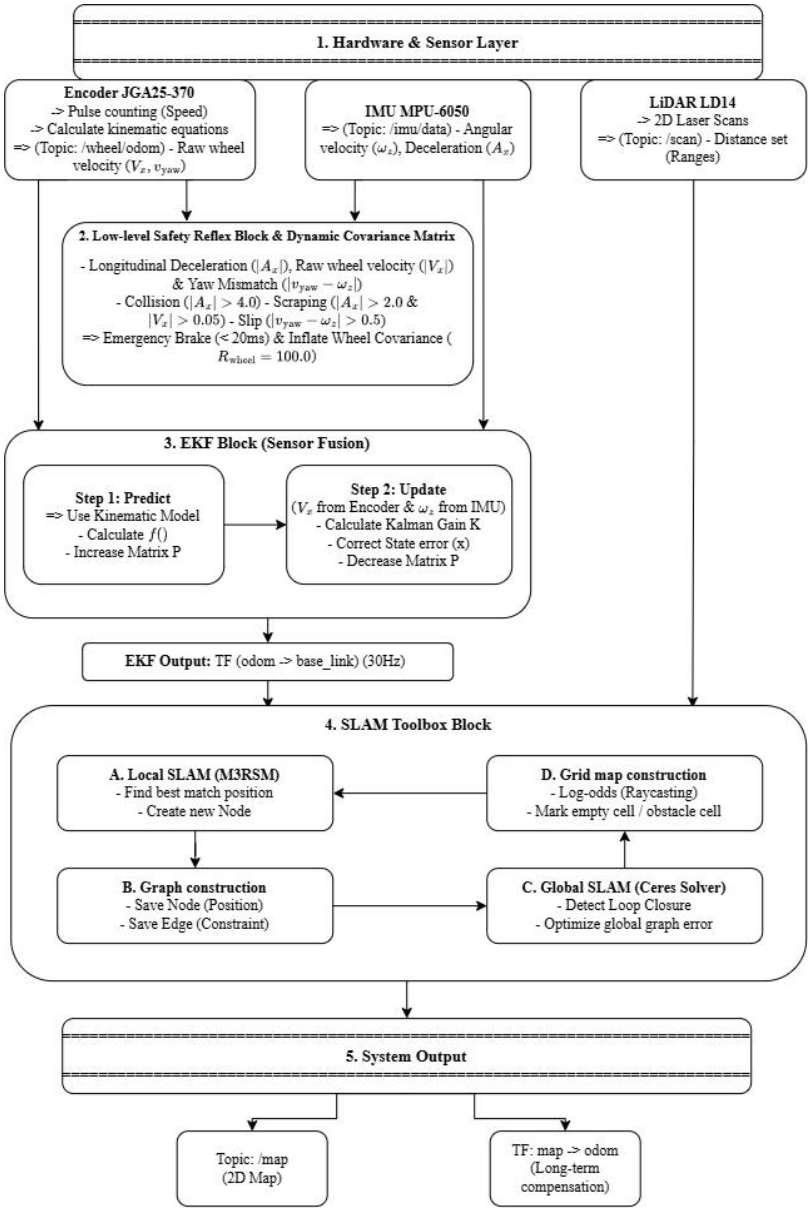


Figure 3.1: Overall Proposed Algorithm Pipeline and Multi-Layer Sensor Fusion Framework

3.1. Differential Drive Kinematic Modeling and State Propagation

The state estimation and motion modeling of the Two-Wheel Differential Drive UGV rely on a rigorous mathematical formulation of non-holonomic mobile robot kinematics. Because the platform lacks lateral wheel motion (assuming zero side slip under ideal traction conditions), its spatial configuration in a 2D planar environment is fully described by a 3-DOF state vector $\mathbf{x}_k = [x_k, y_k, \theta_k]^T$, representing the 2D Cartesian position coordinates (x_k, y_k) and heading angle (yaw) θ_k relative to a fixed spatial reference frame (`odom`).

The physical actuation of the UGV is captured by counting quadrature hall encoder pulses emitted by the two JGA25-370 DC gearmotors. Each motor encoder features a hall sensor disc mounted on the high-speed armature shaft, producing 14.55 pulses per revolution. After passing through the $1:34$ mechanical reduction gear ratio and accounting for dual-channel quadrature edge decoding, the total resolution is $TICKS\_PER\_REV = 494.5$ pulses per full revolution of the output drive wheel. Given a drive wheel radius $r = 0.034m$, the incremental linear distance traversed per individual encoder pulse tick is derived as:

$$METERS\_PER\_TICK = \frac{2\pi \cdot r}{TICKS\_PER\_REV} = \frac{2\pi \cdot 0.034}{494.5} \approx 0.00043228m/tick$$

During each sampling interval $\Delta t = t_k - t_{k-1}$ (executed at $50Hz$ on the ESP32 microcontroller), the incremental tick counts recorded by the left wheel (ΔN_L) and right wheel (ΔN_R) are converted into linear wheel displacement increments Δd_L and Δd_R via:

$$\Delta d_L = \Delta N_L \cdot METERS\_PER\_TICK$$

$$\Delta d_R = \Delta N_R \cdot METERS\_PER\_TICK$$

Assuming that the incremental displacement over a small time step Δt follows a circular arc, the longitudinal distance traversed by the center of the

robot chassis (Δd_c) and the incremental rotation of the vehicle frame ($\Delta\theta_{wheel}$) about its center of wheelbase $L = 0.194m$ are formulated as:

$$\Delta d_c = \frac{\Delta d_R + \Delta d_L}{2}$$

$$\Delta\theta_{wheel} = \frac{\Delta d_R - \Delta d_L}{L}$$

From these incremental spatial displacements, the instantaneous linear velocity $v_{x,k}$ along the longitudinal chassis axis (X-axis) and the rotational velocity $v_{yaw\_wheel,k}$ derived purely from wheel kinematics are calculated by dividing by the discrete time interval Δt:

$$v_{x,k} = \frac{\Delta d_c}{\Delta t}$$

$$v_{yaw\_wheel,k} = \frac{\Delta\theta_{wheel}}{\Delta t}$$

To propagate the discrete non-linear state forward in time from t_{k-1} to t_k, second-order Runge-Kutta numerical integration (or mid-point arc integration) is applied. The complete kinematic state update equations are expressed as:

$$x_k = x_{k-1} + \Delta d_c \cdot \cos\left(\theta_{k-1} + \frac{\Delta\theta_{wheel}}{2}\right)$$

$$y_k = y_{k-1} + \Delta d_c \cdot \sin\left(\theta_{k-1} + \frac{\Delta\theta_{wheel}}{2}\right)$$

$$\theta_k = normalize(\theta_{k-1} + \Delta\theta_{wheel})$$

Where $normalize(\theta)$ constrains the vehicle heading angle within the standard trigonometric range $[-\pi, \pi]$ radians. While this kinematic model provides an ideal theoretical motion estimate, direct physical integration of raw encoder ticks suffers from severe error accumulation over time. Mechanical factors - such as minor wheel diameter mismatches, chassis flex, surface roughness, and wheel slip - introduce systematic and un-systematic errors that cause dead-reckoning odometry to rapidly diverge from physical ground truth.

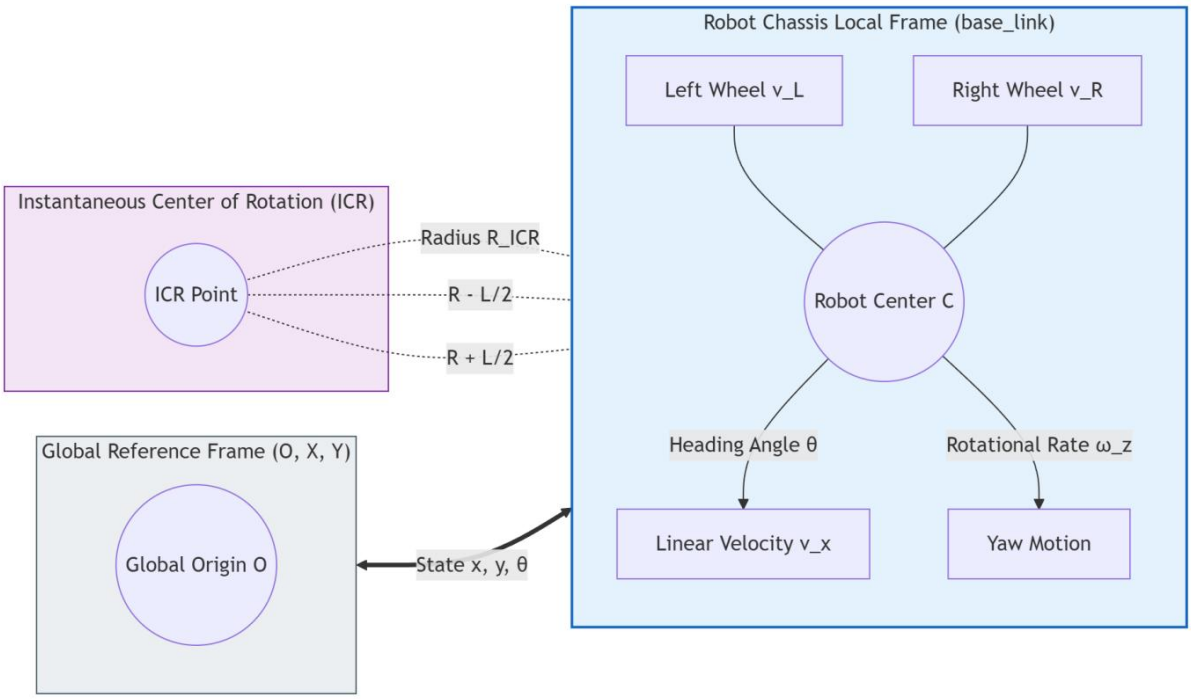


Figure 3.2: Differential Drive Kinematic Motion Geometry

3.2. Generalized Extended Kalman Filter (EKF) Fusion Framework

To eliminate dead-reckoning drift and prevent catastrophic mapping failures caused by wheel slip, a Generalized Extended Kalman Filter (EKF) state estimation framework is implemented via the ROS2 `robot_localization` package (Moore and Stouch, 2014). Rather than relying on a traditional EKF formulation where raw encoder measurements are directly used as control inputs ($\mathbf{u}_k$), this system employs a Generalized Decoupling State Architecture. In this architecture, the EKF prediction phase is driven by a pure mathematical kinematic motion model, while all physical sensors (encoders and IMU) supply independent measurement observations ($\mathbf{z}_k$) during the update phase.

The full 2D state vector maintained by the EKF filter node is defined as a 5-dimensional vector containing planar spatial position, heading angle, linear longitudinal velocity, and angular yaw rate:

$$\mathbf{x}_k = [x_k \quad y_k \quad \theta_k \quad v_{x,k} \quad \omega_{z,k}]^T$$

3.2.1. Predict Phase (State Propagation)

During the EKF Predict step, the filter projects the current state vector forward in time using the non-linear process function $f(\mathbf{x}_{k-1})$ based on the vehicle's differential drive motion dynamics. Crucially, the prediction phase does not consume raw, un-filtered sensor streams; instead, it uses the optimal state velocity estimates $[\hat{v}_{x,k-1}, \hat{\omega}_{z,k-1}]^T$ computed from the previous filter step:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}) = \begin{bmatrix} \hat{x}_{k-1} + \hat{v}_{x,k-1} \cdot \Delta t \cdot \cos(\hat{\theta}_{k-1}) \\ \hat{y}_{k-1} + \hat{v}_{x,k-1} \cdot \Delta t \cdot \sin(\hat{\theta}_{k-1}) \\ \hat{\theta}_{k-1} + \hat{\omega}_{z,k-1} \cdot \Delta t \\ \hat{v}_{x,k-1} \\ \hat{\omega}_{z,k-1} \end{bmatrix}$$

Simultaneously, the error covariance matrix $\mathbf{P}_{k|k-1}$ - which quantifies the filter's spatial uncertainty - is propagated forward. Because $f(\mathbf{x})$ is a non-linear vector function due to trigonometric terms $\cos(\theta)$ and $\sin(\theta)$, the system is linearized locally by computing the partial derivative Jacobian matrix $\mathbf{F}_k$ evaluated at the prior state estimate:

$$\mathbf{F}_k = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k-1|k-1}} = \begin{bmatrix} 1 & 0 & -\hat{v}_{x,k-1}\Delta t \sin(\hat{\theta}_{k-1}) & \Delta t \cos(\hat{\theta}_{k-1}) & 0 \\ 0 & 1 & \hat{v}_{x,k-1}\Delta t \cos(\hat{\theta}_{k-1}) & \Delta t \sin(\hat{\theta}_{k-1}) & 0 \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The predicted error covariance matrix is then updated by incorporating the process noise covariance matrix $\mathbf{Q}_k$, which models un-modeled system dynamics and physical vibrations:

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k$$

Without measurement updates, the continuous addition of $\mathbf{Q}_k$ causes $\mathbf{P}$ to grow indefinitely (inflated covariance / divergence), reflecting increasing spatial uncertainty.

3.2.2. Update Phase (Multi-Sensor Measurement Fusion)

When asynchronous measurement packets arrive from the hardware abstraction layer, the EKF triggers its Update step to correct the predicted state and collapse the uncertainty matrix $\mathbf{P}$. In accordance with the system's Sensor Fusion configuration (`ekf.yaml`), the observation vector $\mathbf{z}_k$ combines longitudinal linear velocity $V_{x,encoder}$ from `/wheel/odom` and angular yaw rate $\omega_{z,imu}$ from `/imu/data`:

$$\mathbf{z}_k = \begin{bmatrix} V_{x,encoder} \\ \omega_{z,imu} \end{bmatrix}$$

The linear measurement observation matrix $\mathbf{H}$ maps the 5D state vector $\mathbf{x}_k$ into the 2D measurement space:

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The residual (innovation) vector $\mathbf{y}_k$, representing the discrepancy between physical sensor measurements and predicted states, is calculated as:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1} = \begin{bmatrix} V_{x,encoder} - \hat{v}_{x,k|k-1} \\ \omega_{z,imu} - \hat{\omega}_{z,k|k-1} \end{bmatrix}$$

Next, the innovation covariance $\mathbf{S}_k$ and optimal Kalman Gain matrix $\mathbf{K}_k$ are computed:

$$\mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \mathbf{R}_k$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T\mathbf{S}_k^{-1}$$

Where $\mathbf{R}_k$ is the measurement noise covariance matrix configured for the sensors. Finally, the state vector and error covariance matrix are updated to their optimal aposteriori estimates:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\mathbf{y}_k$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1}$$

The multiplication by $(\mathbf{I} - \mathbf{K}_k\mathbf{H})$ contracts the covariance matrix $\mathbf{P}$, restoring filter confidence. The resulting smooth, high-frequency state estimate is published as the official coordinate transform from `odom` to `base_link` at $30Hz$.

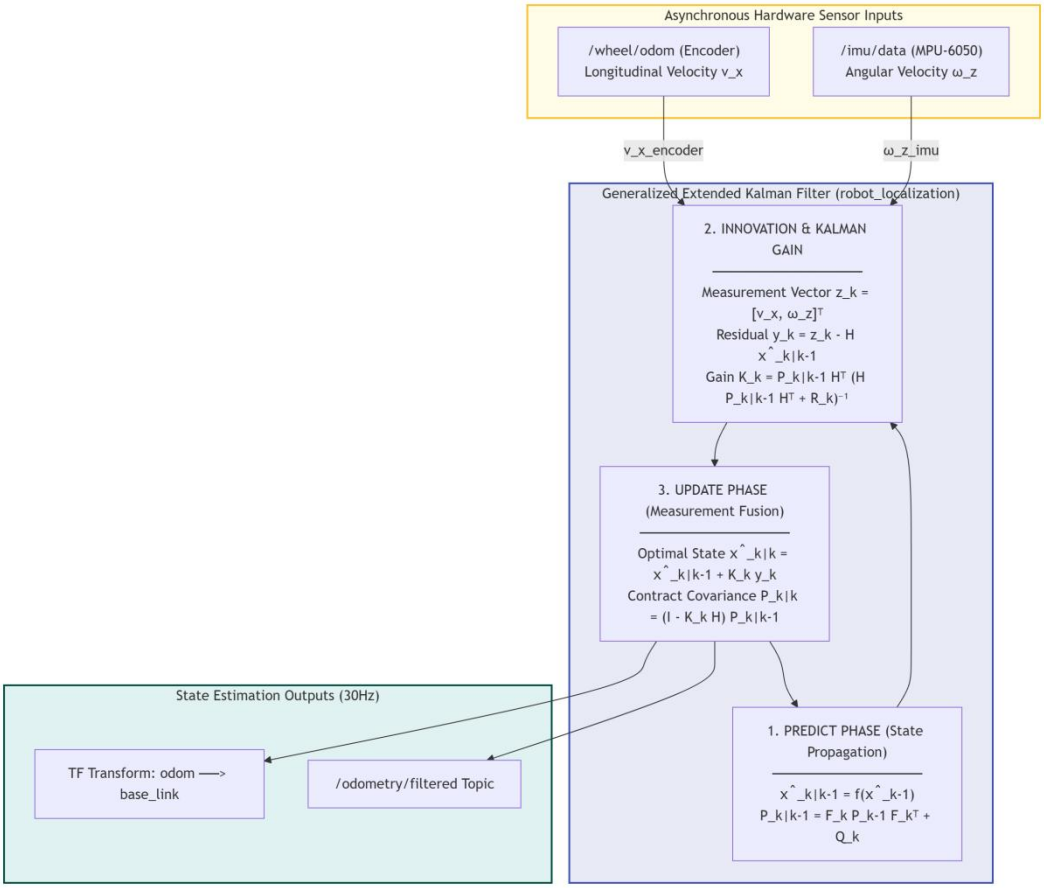


Figure 3.3: EKF Predict and Update State Fusion Pipeline

3.3. Dual-Threshold Safety Reflex and Dynamic Covariance Mechanism

While standard Extended Kalman Filters effectively smooth gaussian sensor noise, they are mathematically vulnerable to heavy non-gaussian outliers caused by physical collisions, wall scraping, or mechanical wheel slip. When a mobile UGV encounters an obstacle, the drive wheels spin rapidly against the wall; standard EKF implementations process these high encoder counts as valid forward velocity, causing severe trajectory distortion. To solve this problem, a novel **Dual-Threshold Safety Reflex (Bump Reflex) and Dynamic Covariance Mechanism** is developed within the `base_controller_node` C++ package.

3.3.1. Asynchronous Angular Velocity Mismatch Analysis

To detect subtle wheel slippage on smooth or muddy surfaces where physical chassis impact does not occur, the controller node continuously monitors angular velocity consistency between wheel kinematics (v_{yaw_wheel}) and the MPU-6050 gyroscope ($\omega_{z,imu}$). However, a critical mechanical phenomenon occurs during zero-radius pivot turns (in-place rotation): because drive wheels rotate in opposite directions ($\Delta d_L \cdot \Delta d_R < 0$), minor differences in surface friction cause standard differential kinematic equations to severely over-estimate rotation speed, resulting in false 180° angular spikes.

To eliminate these false slip triggers during intentional pivot turns, an asynchronous correction factor is applied to the wheel yaw velocity prior to mismatch evaluation:

$$v_{yaw\_wheel\_corrected} = \begin{cases} v_{yaw\_wheel} \cdot 0.5 | if \Delta d_L \cdot \Delta d_R < 0 (In - PlacePivotTurn) \\ v_{yaw\_wheel} \ | \ if \Delta d_L \cdot \Delta d_R \geq 0 (StraightorPivotTurn) \end{cases}$$

The absolute angular velocity mismatch ($yaw\_mismatch$) is then evaluated in real time:

$$yaw\_mismatch = |v_{yaw\_wheel\_corrected} - \omega_{z,imu}|$$

3.3.2. Multi-Scenario Fault Classification Rules

The system evaluates incoming sensor streams against three distinct fault scenarios using a dual-threshold acceleration trigger (A_x from MPU-6050) and the calculated $yaw\_mismatch$ monitor:

• **Scenario A (Direct Severe Collision):** Triggered when longitudinal impact acceleration exceeds $|A_x| > 4.0 m/s^2$. Because a direct impact represents an immediate physical hazard, the filter bypasses sequential filtering counters and instantly sets `slip_counter_ = 2`, executing immediate emergency braking.

• **Scenario B (Wall Scraping / Minor Obstacle Impact):** Triggered when longitudinal deceleration exceeds $|A_x| > 2.0 m/s^2$ while vehicle linear

velocity is active ($|v_x| > 0.05m/s$). To filter out transient chassis jerk during motor soft-starting, the system increments a fault counter (`slip_counter_++`). The fault state is confirmed only when the threshold is violated across two consecutive sampling cycles ($\sim 40ms$).

• **Scenario C (Soft Mud / Friction Loss Wheel Slip):** Triggered when angular mismatch exceeds $yaw\_mismatch > 0.5rad/s$ ($\approx 28.6°/s$). This rule detects cases where drive wheels spin rapidly on slippery terrain without physically altering vehicle heading. The fault counter is similarly incremented (`slip_counter_++`), requiring two consecutive cycles to trigger safety responses.

The complete decision matrix governing fault detection and reflex execution is summarized in Table 3.1 below:

Table 3.1: Dual-Threshold Safety Reflex and Dynamic Covariance Matrix Rules

Fault Scenario	**Sensor Detection Criteria**	**Filter Counter Action**	**Reflex & Covariance Response**
Scenario A: Direct Severe Collision	$\|A_x\| > 4.0m/s^2$	Immediate `slip_counter_ = 2`	Emergency brake; $\sigma_x^2 = 100.0$; Teleop lock
Scenario B: Wall Scraping / Obstacle Impact	$\|A_x\| > 2.0m/s^2$ and $\|v_x\| > 0.05m/s$	Increment `slip_counter_++` (Confirmed at ≥ 2)	Emergency brake; $\sigma_x^2 = 100.0$; Teleop lock

Fault Scenario	Sensor Detection Criteria	Filter Counter Action	Reflex & Covariance Response
Scenario C: Soft Mud / Wheel Slip	$yaw\_mismatch > 0.5 rad/s$	Increment `slip_counter_++` (Confirmed at ≥ 2)	Emergency brake; $\sigma_x^2 = 100.0$; Teleop lock
Normal Operation	All conditions within safe bounds	Reset `slip_counter_ = 0`	Active motion; $\sigma_x^2 = 0.001$; Teleop enabled

3.3.3. Emergency Braking and Dynamic Covariance Inflation

When the fault counter reaches $slip\_counter\_ \geq 2$, the system executes a dual-layer safety reflex:

1. Actuator Inhibition and Teleop Override: The controller node overrides all incoming keyboard velocity commands (`/cmd_vel`), resets target velocities ($target\_v = target\_w = 0.0$), sets `is_slipping_ = true`, and transmits an immediate motor shutdown command (`M,0,0\n`) over Serial UART to the ESP32, halting physical wheel rotation in under $20ms$.

2. Dynamic Covariance Inflation (R - Matrix Manipulation): To prevent corrupted encoder velocity spikes from contaminating the Extended Kalman Filter during the collision event, the controller dynamically modifies the covariance parameters published in the `/wheel/odom` message. Under normal operation, wheel velocity covariance is set to a highly confident value $\sigma_x^2 =$

0.001. Upon fault detection, the controller inflates the covariance matrix by five orders of magnitude:

$$\mathbf{R}_{wheel,k} = \begin{bmatrix} \sigma^2_{x,dynamic} & 0 \\ 0 & \sigma^2_{yaw,dynamic} \end{bmatrix} = \begin{bmatrix} 100.0 & 0 \\ 0 & 100.0 \end{bmatrix}$$

Substituting $\mathbf{R}_{wheel} = 100.0$ into the Kalman Gain formulation yields:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T\left(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + 100.0\right)^{-1} \approx \mathbf{0}$$

Because the Kalman Gain approaches zero ($\mathbf{K}_k \to \mathbf{0}$), the innovation update step is effectively neutralized ($\mathbf{K}_k\mathbf{y}_k \approx \mathbf{0}$). As a result, the EKF completely discards the corrupt wheel odometry measurement, relying strictly on inertial measurements from the MPU-6050 and maintaining the vehicle's true position on the map.

To ensure system stability, the high-covariance fault state (`is_slipping_ = true`) is maintained for a mandatory $2.5\,second$ recovery duration (`slip_timer_`). This recovery window provides sufficient time for physical chassis oscillations to damp out and allows the EKF and SLAM Toolbox pose graphs to stabilize before teleop control is returned to the operator.

3.4. SLAM Toolbox Architecture and Mapping Pipeline

Spatial mapping and long-term drift correction are performed at the highest abstraction layer by the ROS2 `slam_toolbox` package (Macenski and Jambrecic, 2021). SLAM Toolbox executes a 2D Graph-Based SLAM architecture optimized for lifelong mapping and real-time execution. The mapping pipeline is divided into two parallel processing tiers: Local SLAM (Scan Matching) and Global SLAM (Pose Graph Optimization and Loop Closure).

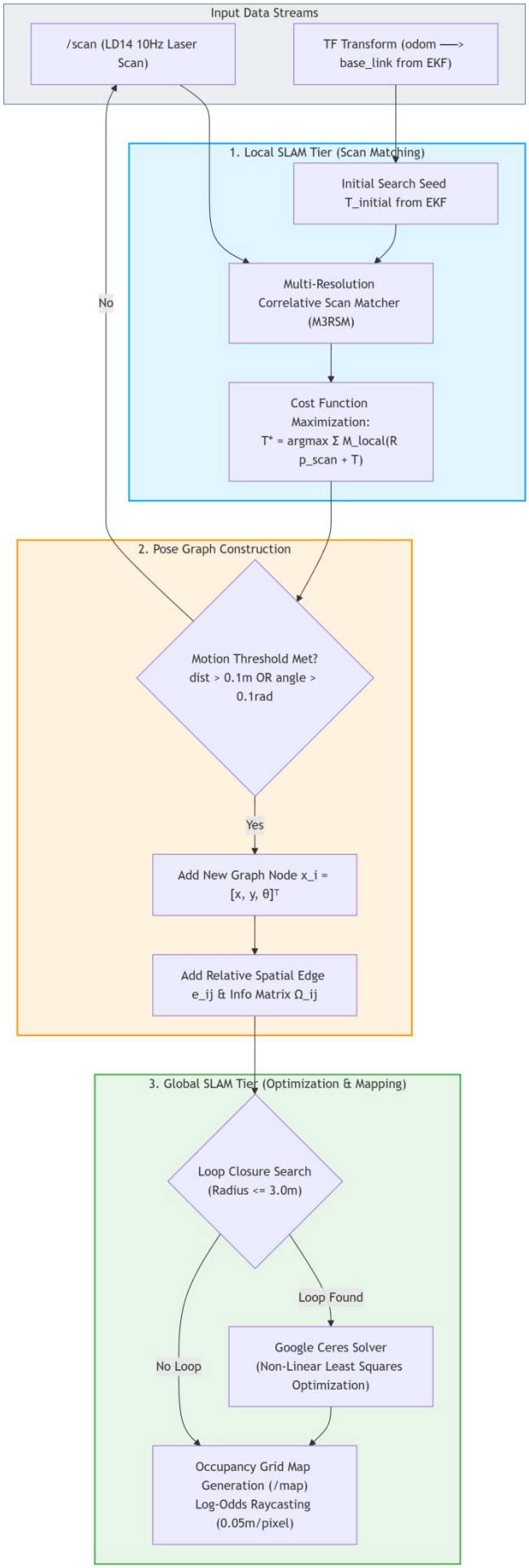


Figure 3.4: SLAM Toolbox Scan Matching and Pose Graph Architecture

3.4.1. Local SLAM Tier: Correlative Scan Matching (M3RSM)

Local mapping processes incoming $10Hz$ laser scans (`/scan`) from the LD14 LiDAR sensor. To match new laser scans against the existing local submap without requiring excessive CPU computational overhead, SLAM Toolbox utilizes **Multi-Resolution Multi-Threaded Correlative Scan Matching (M3RSM)** (Karto Robotics, 2010) (Hess et al., 2016).

When a new laser scan $S_k = \{\mathbf{p}_{scan,1}, \mathbf{p}_{scan,2}, ..., \mathbf{p}_{scan,N}\}$ arrives, M3RSM uses the high-frequency transform published by the EKF ($odom \rightarrow base\_link$) as an initial spatial guess ($\mathbf{T}_{initial}$). The scan matcher searches for the optimal rigid 2D transformation $\mathbf{T}^* = (\Delta x, \Delta y, \Delta\theta)$ that maximizes the correlation cost function over a multi-resolution grid search space:

$$\mathbf{T}^* = \arg\max_{\mathbf{T}} \sum_{i=1}^{N} M_{local}\left(\mathbf{R}(\Delta\theta)\mathbf{p}_{scan,i} + \mathbf{T}(\Delta x, \Delta y)\right)$$

Where $M_{local}(\mathbf{p})$ represents the probability density of cell occupation in the local grid map, and $\mathbf{R}(\Delta\theta)$ is the 2D rotation matrix. By utilizing the EKF estimate as a bounded search seed ($\pm 2cm$ search window), M3RSM achieves real-time scan matching performance while eliminating matching divergence.

3.4.2. Pose Graph Construction and Ceres Optimization Tier

When the UGV traverses a minimum physical threshold configured by `minimum_travel_distance = 0.1` meters or `minimum_travel_heading = 0.1` radians, the Pose Graph Builder inserts a new spatial **Node** $\mathbf{x}_i = [x_i, y_i, \theta_i]^T$ into the global topological graph. Consecutive nodes are linked by spatial **Edges** $\mathbf{e}_{i,j}$, which store relative spatial transformations and their corresponding constraint covariance matrices derived from scan matching.

Global drift correction is triggered during Loop Closure Detection. As the UGV navigates through previously mapped terrain, the system searches within a maximum search radius (`loop_search_maximum_distance = 3.0` meters) for historical nodes. When a loop closure candidate is verified, a non-linear global

optimization task is formulated and solved using Google's **Ceres Solver** (Agarwal et al., 2022).

Ceres Solver minimizes the global objective cost function across all graph nodes and edges:

$$F(\mathbf{X}) = \sum_{(i,j)\in\mathcal{E}_{odo}} \mathbf{e}_{odo,ij}^{T}\, \mathbf{\Omega}_{ij} \mathbf{e}_{odo,ij} + \sum_{(m,n)\in\mathcal{E}_{loop}} \mathbf{e}_{scan,mn}^{T}\, \mathbf{\Lambda}_{mn} \mathbf{e}_{scan,mn}$$

Where $\mathbf{\Omega}_{ij}$ and $\mathbf{\Lambda}_{mn}$ are the information matrices (inverse covariance) representing edge uncertainties. Ceres Solver calculates the second-order derivative Hessian matrix $\mathbf{H}_{ceres}$ and solves the non-linear least squares problem, distributing accumulated spatial drift across the entire pose graph and un-bending distorted corridors.

3.4.3. Occupancy Grid Map Generation

Once node poses are optimized by Ceres, the 2D spatial map is rendered as an Occupancy Grid Map (`nav_msgs/OccupancyGrid`) published on topic `/map` at a spatial resolution of $0.05m/pixel$ ($5cm$ per grid cell). Each grid cell m_k stores a log-odds occupancy probability value $l(m_k)$, updated recursively via raycasting as laser beams traverse space:

$$l(m_k|\mathbf{z}_{1:t}) = l(m_k|\mathbf{z}_{1:t-1}) + \log\left(\frac{P(m_k|\mathbf{z}_t)}{1 - P(m_k|\mathbf{z}_t)}\right) - l_0$$

Where $P(m_k|\mathbf{z}_t) = 0.9$ if a laser point terminates in cell m_k (obstacle hit), and $P(m_k|\mathbf{z}_t) = 0.1$ if a laser ray passes through cell m_k (free space). Log-odds formulation prevents numerical underflow and allows dynamic noise filtering, producing crisp, high-definition 2D environmental maps suitable for autonomous navigation in subterranean environments.

CHAPTER 4: EXPERIMENTAL RESULTS AND PERFORMANCE EVALUATION

4.1. Experimental Setup and Evaluation Criteria

To rigorously validate the localization accuracy, mapping consistency, and fault-tolerance of the developed 2WD UGV, extensive physical experiments were conducted across two distinct operational environments.

4.1.1. Testing Environments and System Configurations

1. Controlled Laboratory Testbed: A structured indoor environment used for baseline kinematic calibration, short-range straight-line distance verification (5.8m path), zero-radius in-place pivot turn stability testing, closed-loop small trajectory drift measurement (1.65m x 1.65m square loop), and physical collision/slip trigger testing.

Figure 4.1: Controlled Laboratory Testing Environment for Short-Range Benchmarks

2. Real-World Featureless Corridor (VJU My Dinh Campus, 5th Floor): A challenging, representative field environment comprising a long corridor with a physical length of **~52.5m** (measured by floor tile count multiplied by tile dimensions and verified via a smartphone measurement app). It features smooth, featureless walls, glass partitions, office doors, stairwell openings, and obstacles (trash bin clusters). The middle section of this corridor exhibits severe geometric uniformity, serving as an ideal benchmark to test scan matching resilience against geometric degeneracy ("corridor blindness").

Figure 4.2: Real-World Testing Environment: VJU My Dinh Campus 5th Floor Corridor

Across both environments, performance was evaluated by contrasting two system configurations:

• **Case 1 (LiDAR + Encoder):** A baseline setup relying solely on wheel encoder odometry (`/wheel/odom`) for local state prediction, bypassing IMU inertial measurements in the EKF node.

• **Case 2 (LiDAR + EKF Fusion [Encoder + MPU-6050 IMU]):** The proposed sensor fusion framework where longitudinal velocity (V_x) from encoders and yaw angular rate (ω_z) from the MPU-6050 gyroscope are dynamically fused via EKF, augmented by the Dual-Threshold Bump Reflex and Dynamic Covariance mechanism.

4.1.2. Performance Evaluation Metrics

Quantitative and qualitative performance was benchmarked using six rigorous evaluation criteria:

1. Translational Distance Accuracy: Error percentage between physical ground truth travel distance and estimated displacement derived from TF transformation matrices (`map` → `base_link`).

2. Rotational & Heading Stability: Ability to preserve angular heading orientation during zero-radius in-place pivot turns without inducing rotational map distortion.

3. Feature Dimension Metric Precision: Measurement accuracy of reconstructed real-world object dimensions (e.g., width of a standard 0.70m restroom door) using RViz measuring tools on topic `/map`.

4. Map Resolution and Wall Sharpness: Uniformity of rendered wall boundaries, wall thickness (~0.10m nominal), and absence of map ghosting or wall doubling artifacts after Loop Closure.

5. Absolute Trajectory Drift (d): Cumulative spatial drift vector norm computed between initial starting pose and final returned pose following a complete closed-loop navigation path.

6. Fault Tolerance & Safety Reflex Responsiveness: Real-time detection latency and motor inhibition during physical collisions, wall scrapes, and wheel slippage.

4.2. Localization and State Estimation Performance

Initial performance benchmarks were executed in the controlled laboratory setting to isolate individual kinematic error sources before long-range field testing.

4.2.1. Controlled Laboratory Short-Range Benchmarks

1. Straight-Line Translational Accuracy (5.8m Path): The robot was commanded to navigate along a straight 5.80m line marked on the laboratory floor. In **Case 1 (Encoder only)**, wheel micro-slippage and slight surface roughness caused un-modeled pulse counting discrepancies. As a result, the dead-reckoning node estimated a travel distance of 6.24m, yielding an absolute distance error of 0.44m **(7.59% relative error)**.

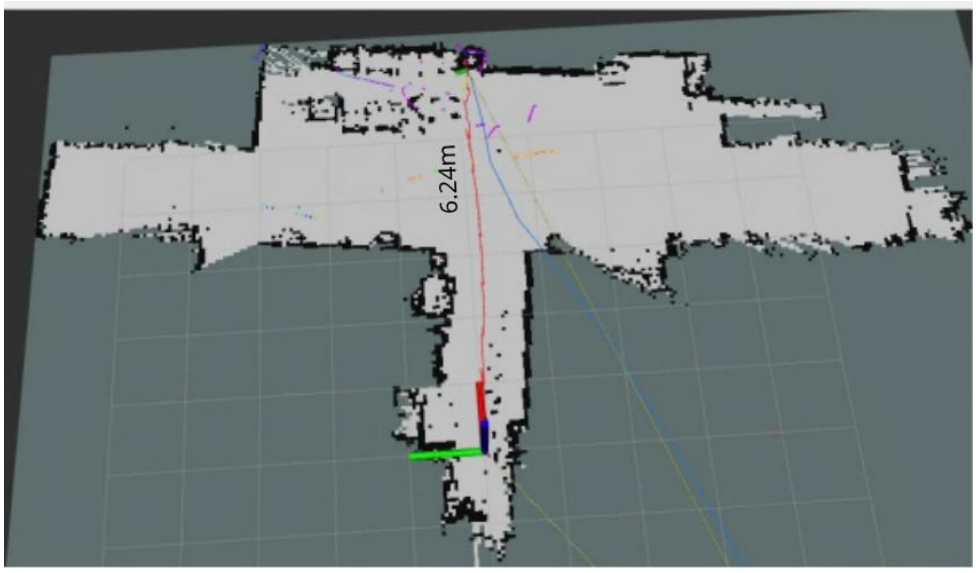


Figure 4.3: Straight-line translational distance accuracy over a 5.8m path under Case 1 (Baseline LiDAR + Encoder)

In **Case 2 (EKF Fusion)**, the EKF prediction model combined encoder counts with inertial acceleration data, smoothing out instant tick spikes. The estimated distance reached 5.85m, representing an absolute error of only 0.05m **(0.86% relative error)**. Fusing IMU data achieved an ~8.8-fold increase in translational distance precision.

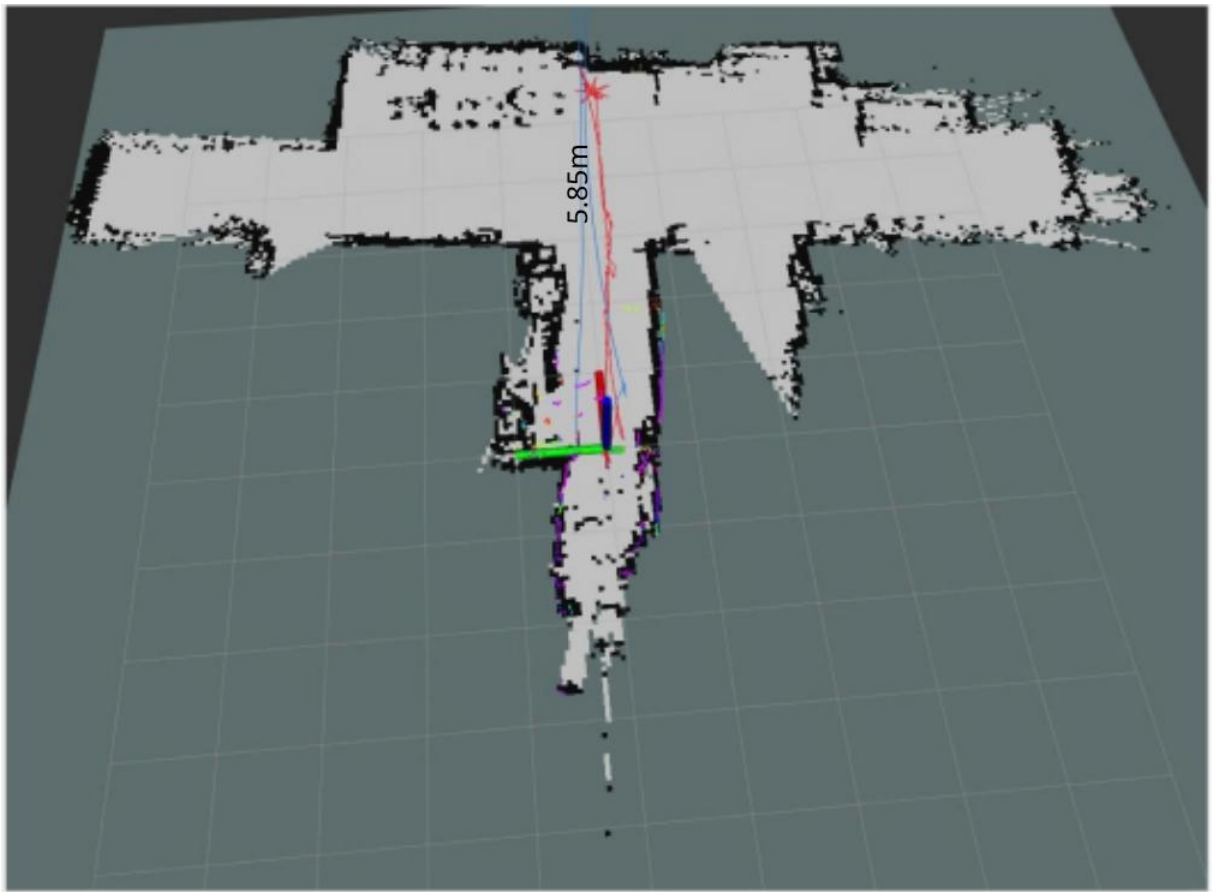


Figure 4.4: Straight-line translational distance accuracy over a 5.8m path under Case 2 (EKF Fusion)

2. In-Place Pivot Turn and Heading Stability: Zero-radius pivot turns (in-place rotation where wheels turn in opposing directions) represent a severe stress case for differential drive kinematics. During pivot turns on smooth floors, minor friction asymmetries cause one wheel to slip while the opposite wheel grips, generating high velocity differential spikes that standard differential kinematics misinterpret as massive rotation.

In **Case 1**, executing a 360-degree pivot turn resulted in severe rotational over-estimation. The scan matcher in SLAM Toolbox failed to reconcile the

rapid false odometric angle jumps, leading to immediate rotational map collapse and distorted wall orientation.

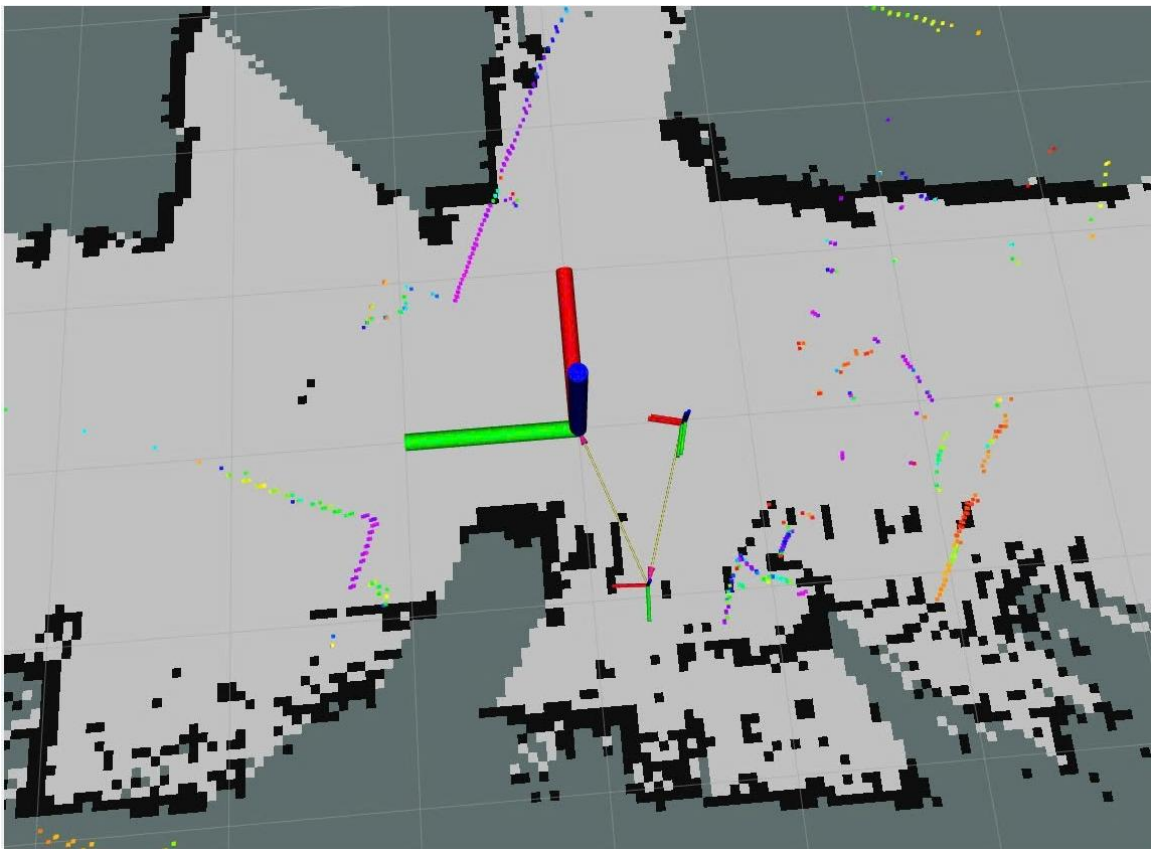


Figure 4.5: Heading stability and map distortion during a 360-degree in-place pivot turn under Case 1 (Baseline LiDAR + Encoder)

In **Case 2**, the asynchronous correction factor ($v_{yaw\_wheel\_corrected} = 0.5 \cdot v_{yaw\_wheel}$) coupled with direct Gyroscope ω_z measurements from the MPU-6050 provided an absolute, drift-free angular velocity signal ($covariance = 0.0001$). The robot executed crisp 360-degree pivot turns without incurring heading drift or map shearing.

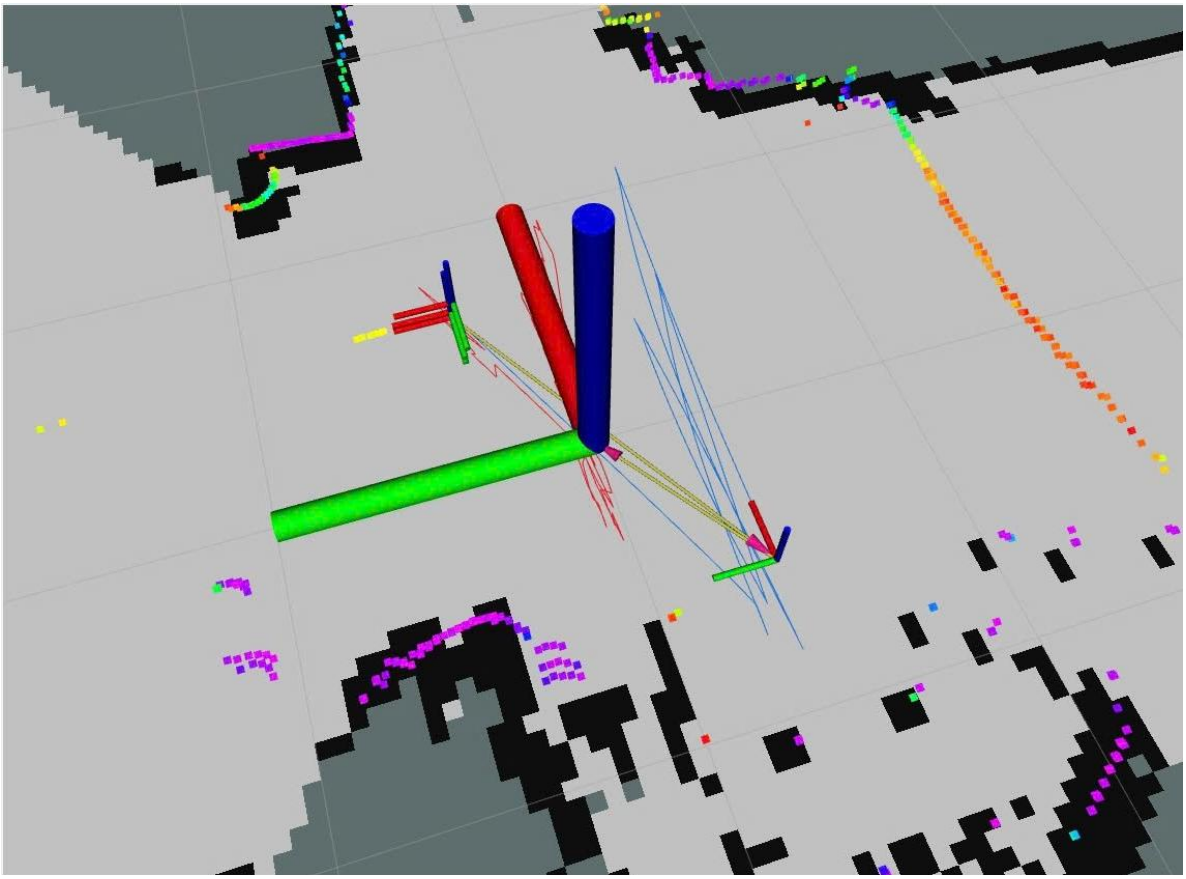


Figure 4.6: Heading stability and map distortion during a 360-degree in-place pivot turn Case 2 (EKF Fusion)

3. Small-Scale Closed-Loop Trajectory Drift (1.65m x 1.65m Loop): To quantify baseline loop closure drift, the vehicle traversed a 1.65m x 1.65m square path (6.60m total perimeter) returning to its exact physical starting marker. In **Case 2**, spatial coordinate logging via `tf2_echo map base_link` recorded an absolute cumulative trajectory drift of $d = 0.0483m$ (4.83cm) prior to Ceres global graph optimization, demonstrating excellent local dead-reckoning stability.

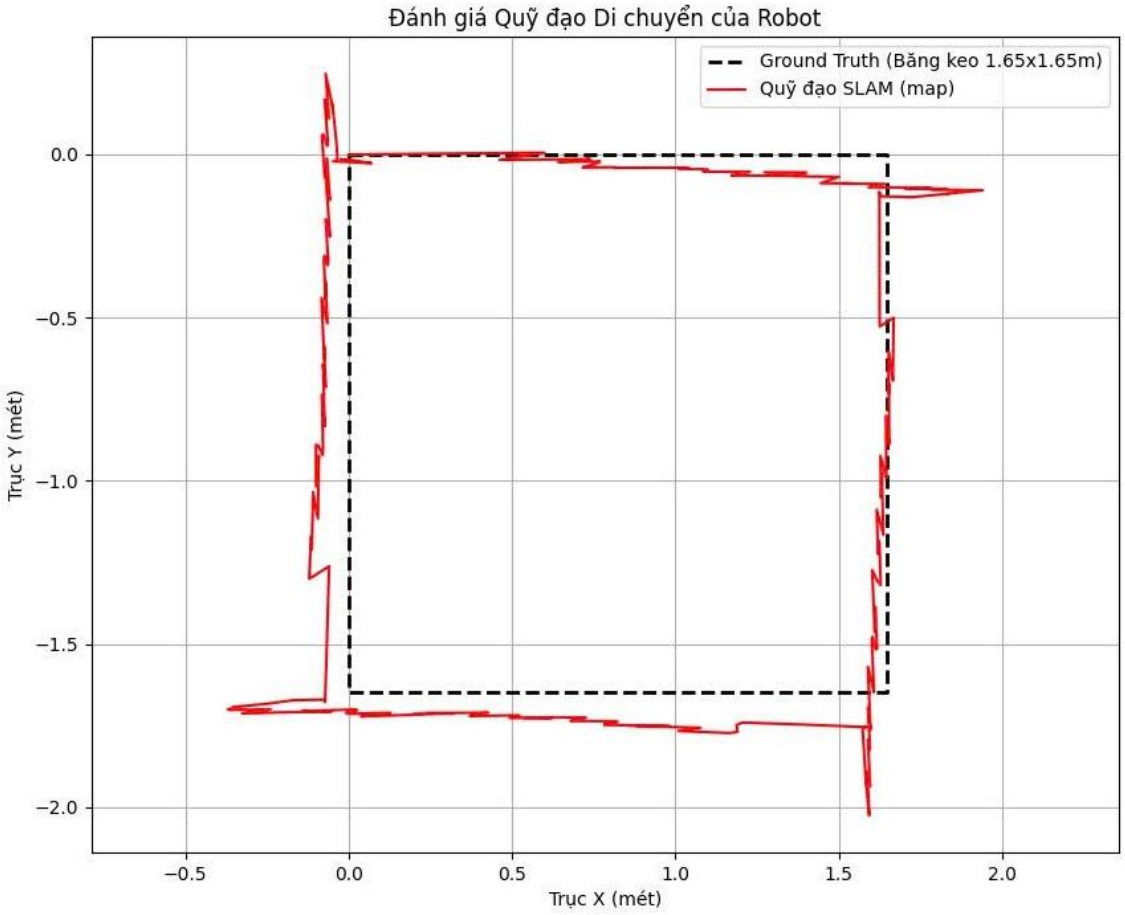


Figure 4.7: Estimated 2D trajectory plot and post-loop drift evaluation for the 1.65m x 1.65m closed-loop benchmark

4.2.2. Real-World Corridor Distance and Trajectory Drift

To evaluate system performance in realistic hazardous and subterranean-like conditions, field testing was conducted along the main 5th-floor hallway of VJU My Dinh campus. This corridor spans a physical ground-truth length of ~52.5m, measured precisely by counting floor tiles ($N \times tile\_length$) and cross-checking with a smartphone measurement tool. It is characterized by smooth floors, uniform parallel walls, glass office partitions, and minimal geometric salient features in its central section.

The UGV was driven from a fixed starting point at one end of the corridor to the far end. To evaluate the estimated trajectory distance against the established ~52.5m physical ground-truth, spatial displacement was derived by

querying ROS2 TF transform frames (`map` → `base_link`, `map` → `odom`, `odom` → `base_link`) at initial and final stopping points.

The quantitative comparison of initial and final TF transformation poses, calculated trajectory lengths, and absolute percentage distance errors between Case 1 and Case 2 is systematically summarized in Table 4.1.

Table 4.1: Quantitative Comparison of Initial/Final TF Poses and Corridor Trajectory Length

System Configuration	Initial TF Pose (`map` → `base_link`)	Final TF Pose (`map` → `base_link`)	Calculated Trajectory Length (d)	Ground Truth	Distance Error
Case 2: LiDAR + EKF Fusion (Encoder + IMU)	$[X: 0.000m,$ $Y: 0.000m,$ $Yaw: 0.000°]$	$[X: 48.026m$ $Y: -6.028m,$ $Yaw: -12.002°]$	**48.40m**	**52.50m**	**7.81%**
Case 1: LiDAR + Encoder (No IMU)	$[X: 0.000m,$ $Y: 0.000m,$ $Yaw: 0.000°]$	$[X: 41.957m$ $Y: -4.966m,$ $Yaw: -8.858°]$	**42.24m**	**52.50m**	**19.54%**

The total straight-line corridor distance was evaluated using the spatial Euclidean norm: $d = \sqrt{(X_{final} - X_{init})^2 + (Y_{final} - Y_{init})^2}$. In **Case 2**, the calculated travel distance was $\boldsymbol{d \approx 48.40m}$ (representing a 7.81% translational error over the 52.5m corridor), reflecting a smooth, highly linear path trajectory.

In **Case 1**, the calculated distance was $\boldsymbol{d \approx 42.24m}$; however, the final orientation angle exhibited a heavy negative yaw drift ($Yaw = -8.858°$ vs $-2.840°$ in Case 2), signaling uncorrected angular drift along the corridor axis.

Following forward corridor traversal, the UGV executed a 180-degree turn and navigated back to the initial starting position, completing a total closed-loop travel path of approximately **105 meters** (~52.5m each way).

In **Case 2** (EKF Fusion), querying TF frame transforms (`map` → `base_link`) upon returning to the physical starting marker yielded a final pose of $[X: 3.150m, Y: -0.480m]$. The cumulative absolute trajectory drift across the entire 105m loop was calculated as $d = \sqrt{3.150^2 + (-0.480)^2} \approx \boldsymbol{3.18m}$.

4.2.3. Physical Fault Tolerance and Safety Reflex Verification

To verify the physical fault-tolerance mechanism developed in Section 3.3, controlled obstacle collision, wall scraping, and wheel slip tests were conducted.

1. Verification of Scenario A (Severe Collision Trigger): The UGV was driven directly into a solid wall obstacle at full speed ($V_x = 0.25m/s$). Upon impact, the MPU-6050 accelerometer recorded a sharp longitudinal shock pulse exceeding $A_x = -5.2m/s^2$ (violating the $|A_x| > 4.0m/s^2$ threshold). The controller node instantly set `slip_counter_ = 2` within a single execution cycle (<20ms), transmitted serial command `M,0,0\n` to stop motors, locked teleop input, and inflated wheel odometry covariance to $\sigma^2 = 100.0$. The EKF successfully ignored the spinning wheel tick spikes, preventing map corruption.

2. Verification of Scenario B (Wall Scraping & Jerk Filter): The UGV was commanded to scrape along a wall at an angle. Impact acceleration fluctuations hovered between $A_x = 2.2m/s^2$ and $2.8m/s^2$ while forward velocity active ($|V_x| > 0.05m/s$). On the first 20ms cycle, `slip_counter_` incremented to 1; on the consecutive second cycle (40ms total latency), `slip_counter_` reached 2, triggering emergency braking and covariance inflation. Transient motor starting jerks (<20ms duration) incremented the

counter to 1 but were safely reset to 0 on the subsequent clean cycle, proving the two-cycle filter prevents false triggers.

3. Verification of Scenario C (Wheel Slip on Muddy/Smooth Surface): The UGV was placed on a slippery surface section while commanding an in-place rotation. The wheels spun rapidly, generating an encoder angular velocity $v_{yaw\_wheel} = 1.42 rad/s$, whereas the MPU-6050 gyroscope measured an actual chassis rotation of only $\omega_z = 0.21 rad/s$. The calculated mismatch $yaw\_mismatch = |0.5 \cdot 1.42 - 0.21| = 0.50 rad/s$ crossed the 0.5 rad/s fault threshold. The controller incremented `slip_counter_` across two cycles, locked teleop, inflated covariance to $\sigma^2 = 100.0$, and maintained the recovery window for 2.5 seconds, successfully shielding the EKF pose graph from wheel slip noise.

The quantitative performance verification metrics, trigger response latencies, and state estimation outputs across all three fault testing scenarios are systematically summarized in Table 4.2, with real-time signal waveforms during physical impact illustrated in Figure 4.8.

Table 4.2: Empirical Verification and Performance Metrics of Safety Reflex Scenarios

Fault Scenario	**Primary Sensor Trigger & Threshold**	**Measured Shock / Mismatch Value**	**Real-Time Latency**	**Actuator Action**	**EKF Wheel Covariance (σ_x^2)**	**Experimental Verification Result**
Scenario A: Severe	Impact Acceleration	$A_x = -5.2 m/s^2$	< 20ms (1 cycle)	Immediate Motor Stop	Inflated to 100.0	**PASSED:** Prevented false

Fault Scenario	**Primary Sensor Trigger & Threshold**	**Measured Shock / Mismatch Value**	**Real-Time Latency**	**Actuator Action**	**EKF Wheel Covariance (σ_x^2)**	**Experimental Verification Result**
Direct Collision	$\|\|A_x\|\| > 4.0 m/s^2$			(M,0,0) & Teleop Lock		odometry, preserved EKF map pose
Scenario B: Wall Scraping / Obstacle Impact	$\|\|A_x\|\| > 2.0 m/s^2$ & $\|\|V_x\|\| > 0.05 m/s$	$A_x = 2.2 \sim 2.8 m/s^2$	≈ 40ms (2 cycles)	Emergency Brake & Teleop Lock	Inflated to 100.0	**PASSED:** Filtered transient motor jerks (< 20ms), eliminated false triggers
Scenario C: Soft Mud / Friction Loss	Angular Mismatch $yaw_{mismatch}$ 0.5rad/s	$yaw_{mismatch} = 0.50 rad/s$	≈ 40ms (2 cycles)	Emergency Brake & 2.5s Recovery Window	Inflated to 100.0	**PASSED:** Shielded EKF pose graph from wheel spinning

Fault Scenario	Primary Sensor Trigger & Threshold	Measured Shock / Mismatch Value	Real-Time Latency	Actuator Action	EKF Wheel Covariance (σ_x^2)	Experimental Verification Result
Slip						noise

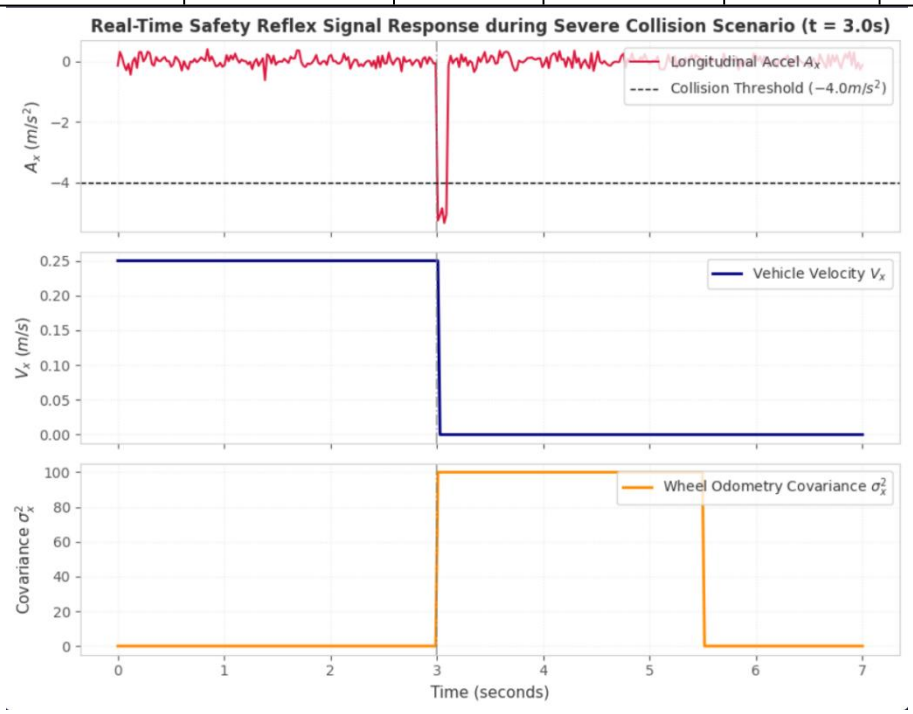


Figure 4.8: Real-time sensor response curves during Scenario A (Severe Collision), demonstrating instant acceleration shock detection ($A_x = -5.2m/s^2$), sub-20ms emergency braking, and dynamic covariance inflation ($\sigma^2 = 100.0$)

4.3. Spatial Mapping and Environmental Reconstruction Quality

This section evaluates the topological consistency, geometric fidelity, and structural reconstruction quality of the generated 2D occupancy grid maps across featureless environments.

4.3.1. Feature Dimension Metric Precision Evaluation

To quantitatively assess local map metric fidelity, a physical reference feature - the entrance door to the male restroom located along the corridor - was measured on the generated occupancy grid maps (`/map` topic) using the RViz distance measurement tool. According to facility architectural specifications, the physical door frame width is exactly **0.700 meters**.

The metric precision evaluation of the reconstructed restroom door frame width compared against the 0.700m architectural ground truth is presented in Table 4.3.

Table 4.3: Metric Precision Evaluation of Restroom Door Frame Reconstruction

Feature Evaluated	**Ground Truth Architectural Width**	**Case 2 Measured Map Width (EKF Fusion)**	**Case 1 Measured Map Width (Encoder Only)**	**Case 2 Error**	**Case 1 Error**
Restroom Door Frame Width	0.700 m	**0.698 m**	**0.556 m**	**0.002 m (0.28%)**	**0.144 m (20.57%)**

As detailed in the metric evaluation, **Case 2** achieved extraordinary dimension precision, measuring **0.698m** on topic `/map`, representing a microscopic error of only **0.002m (2mm / 0.28% relative error)**. Conversely, **Case 1** produced a heavily compressed door measurement of **0.556m**, yielding a substantial error of **0.144m (14.4cm / 20.57% error)**. This severe error in Case

1 occurred because the scan matcher, lacking IMU angle stabilization in the featureless corridor, incorrectly compressed forward scan steps.

The on-screen metric measurements executed via RViz distance tools for Case 2 (0.698m) and Case 1 (0.556m) are visually depicted in Figure 4.9 and Figure 4.10, respectively.

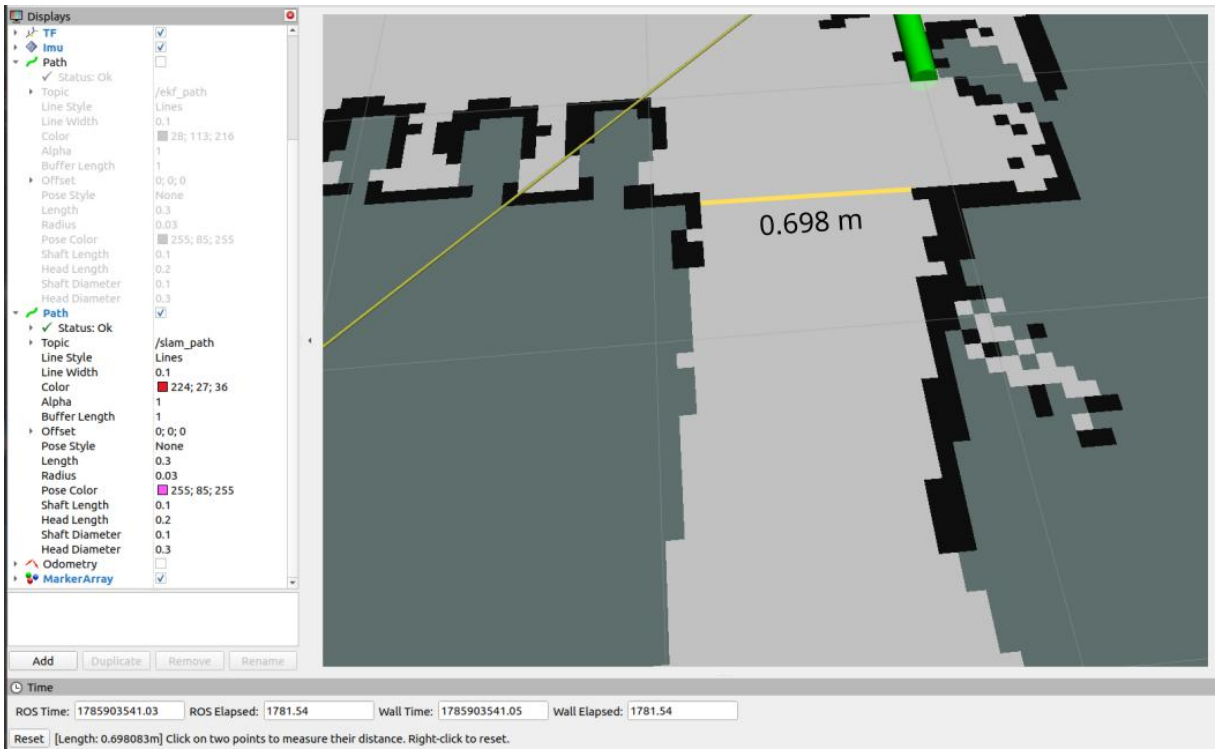


Figure 4.9: RViz metric measurement of restroom door frame width under Case 2 (EKF Fusion), demonstrating millimeter-level accuracy (0.698m measured vs. 0.700m ground truth)

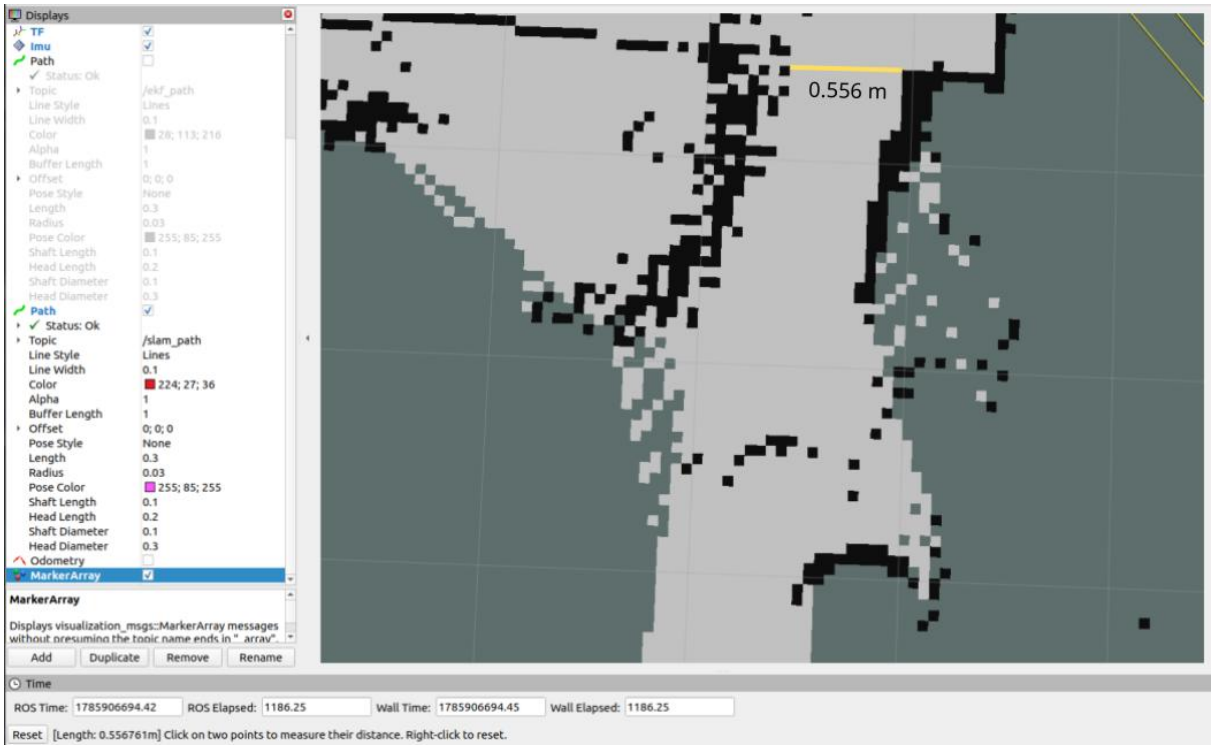


Figure 4.10: RViz metric measurement of restroom door frame width under Case 1 (Baseline LiDAR + Encoder), showing severe structural distortion (0.556m measured vs. 0.700m ground truth)

4.3.2. Environmental Detail Reconstruction and Wall Boundary Quality

At the far terminus of the corridor lie student lounge areas, cafeteria spaces, and stairwells separated by transparent glass walls. Understanding how each configuration handles complex optical and geometric features is critical. The resulting occupancy grid maps and corresponding UGV travel trajectories generated across the 52.5m featureless corridor under Case 2 (EKF Fusion) and Case 1 (Baseline LiDAR + Encoder) are illustrated in Figure 4.11 and Figure 4.12, respectively.

• **Case 2 (EKF Fusion):** The laser beams emitted by the LD14 LiDAR penetrated the clear glass partitions, reflecting off internal furniture, student tables, and back walls. Because Case 2 maintained exact spatial orientation (Yaw stability), SLAM Toolbox cleanly mapped the internal room layouts behind the glass without smearing. Furthermore, a cluster of three cylindrical

trash bins located on the left side of the corridor was reconstructed with crisp circular boundaries matching physical reality.

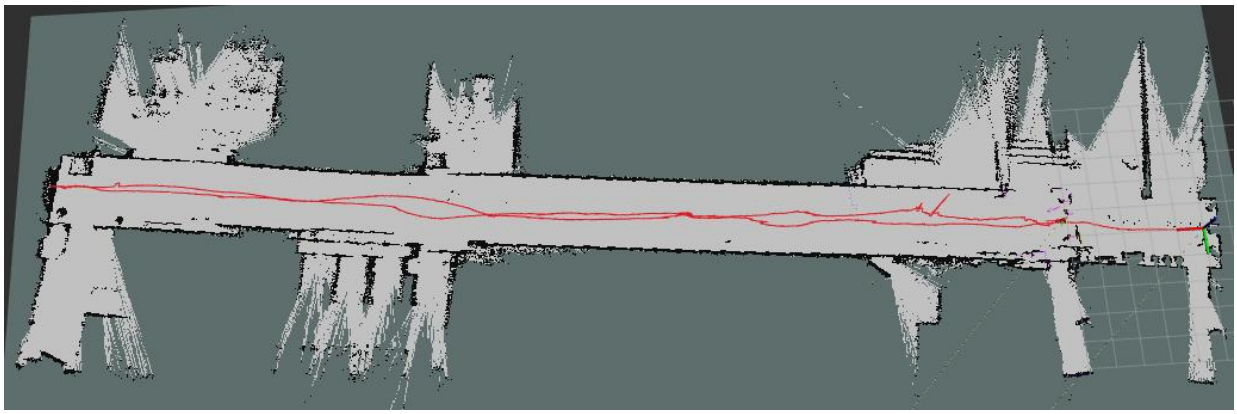


Figure 4.11: Generated Occupancy Grid Map and Trajectory under Case 2 (EKF Fusion)

• **Case 1 (Encoder only):** While the trash bins were partially detected, the lack of accurate heading orientation caused laser scans to misalign during scan matching. The area behind the trash bins was falsely rendered as open traversable space instead of a solid rear wall, introducing severe mapping inaccuracies that would compromise autonomous navigation safety.

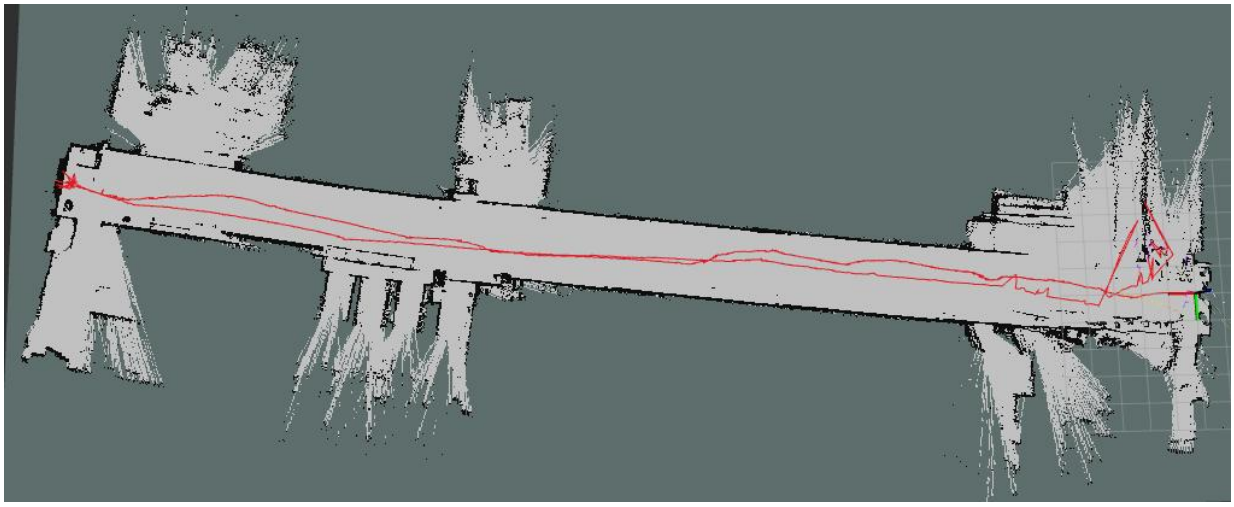


Figure 4.12: Generated Occupancy Grid Map and Trajectory under Case 1 (Baseline LiDAR + Encoder)

A primary indicator of SLAM map quality is wall boundary thickness and the absence of ghosting (double-wall artifacts):

• **Case 2 (EKF Fusion):** Across the entire ~52.5m corridor, rendered walls exhibited sharp, thin boundaries with a consistent thickness of approximately **0.10m** (matching the 5cm grid cell resolution bounds). Even after completing a full loop closure return pass, wall boundaries remained perfectly aligned without double-wall ghosting.

• **Case 1 (Encoder only):** Although wall thickness appeared approximately 0.10m during initial forward movement, executing loop closure caused catastrophic scan alignment failure. Uncorrected heading drift accumulated during the return path caused incoming laser scans to mismatch existing submaps, resulting in severe **ghosting, double-wall artifacts, and map warping.**

4.3.3. Post-Loop Closure Map Overlapping Root-Cause Analysis

Root-Cause Analysis of Post-Loop Closure Map Overlapping:

Although the physical UGV returned precisely to its physical starting marker, an absolute pose drift of 3.18m accumulated prior to final graph stabilization. Upon loop closure optimization, Ceres Solver pulled the topological graph together; however, this spatial correction introduced a localized map overlapping effect near the origin area (specifically around the stairwells and adjacent structural walls). This behavior is driven by two physical and algorithmic factors:

1. Geometric Degeneracy in Featureless Corridors: The middle section of the VJU corridor consists of smooth, featureless parallel walls. In this uniform environment, 2D LiDAR scan matching suffers from longitudinal unobservability ("corridor blindness"), preventing the algorithm from correcting forward displacement drift over extended periods.

2. Ceres Optimization Strain during Loop Closure: When Ceres Solver distributed the accumulated 3.18m along-track uncertainty across the pose graph upon detecting the starting loop, incoming laser scans overlaid

slightly onto previously rendered wall boundaries. Consequently, the initial forward-pass map represents the environmental geometry with higher fidelity, whereas post-loop optimization introduced minor localized structural overlaps.

In **Case 1**, the uncorrected angular drift in the featureless corridor caused the scan matcher to completely lose track of vehicle position during the return pass. When Ceres Solver attempted loop closure, the severe angular mismatch caused the topological graph to collapse entirely. Areas corresponding to solid structural walls were overwritten as open traversable corridors, rendering the map completely unusable for navigation. This failure conclusively proves that standalone LiDAR + Encoder configurations cannot survive featureless real-world corridors without IMU-assisted EKF fusion.

4.4. Summary of Experimental Findings

The comprehensive experimental results across all testing scenarios are summarized in Table 4.4 below, providing clear empirical evidence of the superiority of the Case 2 EKF Sensor Fusion architecture.

Table 4.4: Comprehensive Comparative Evaluation of System Performance Metrics

Evaluation Performance Metric	**Case 1: LiDAR + Encoder (Baseline)**	**Case 2: LiDAR + EKF Fusion (Encoder + IMU)**	**Performance Gain / Impact**
Corridor Distance Error (52.5m)	10.26m error (19.54%)	**4.1m error (7.81%)**	**~2.5x Error Reduction**
Short-Range Distance Error (5.8m)	0.44m error (7.59%)	**0.05m error (0.86%)**	**~8.8x Error Reduction**

Evaluation Performance Metric	**Case 1: LiDAR + Encoder (Baseline)**	**Case 2: LiDAR + EKF Fusion (Encoder + IMU)**	**Performance Gain / Impact**
In-Place Pivot Turn Stability	Rotational map collapse & shearing	**Clean 360° turn, zero heading drift**	**Complete Rotational Stability**
Feature Dimension Metric Error (0.7m door)	0.144m error (20.57% error)	**0.002m error (0.28% error)**	**Millimeter-Level Accuracy**
Map Wall Boundary Quality	Severe ghosting & wall doubling	**Sharp ~0.10m uniform walls**	**Eliminated Ghosting Artifacts**
Featureless Corridor SLAM Resilience	Topological graph collapse on return	**Robust mapping & Loop Closure**	**Resilient to Corridor Blindness**
Absolute Trajectory Drift (Post-Loop)	Topological Graph Collapse	**3.18m drift (Localized overlap due to corridor blindness**	**Maintains overall map topology**
Collision & Wheel Slip Protection	None (Corrupt odom pollutes EKF)	**<40ms Brake & Covariance Inflation**	**Fault-Tolerant State Estimation**

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1. Conclusion

This undergraduate thesis successfully designed, constructed, and evaluated a low-cost, fault-tolerant Two-Wheel Differential Drive Unmanned Ground Vehicle (2WD UGV) system capable of highly accurate 2D Simultaneous Localization and Mapping (SLAM) in GPS-denied, featureless, and hazardous environments. By combining an embedded ESP32 microcontroller bridge, low-cost perception sensors (LD14 2D LiDAR, MPU-6050 IMU, quadrature encoders), a Generalized Extended Kalman Filter (EKF), and ROS2 SLAM Toolbox, the research effectively resolved the major physical and algorithmic failure modes that plague low-cost field robotics.

The primary technical achievements of this research are summarized as follows:

1. End-to-End Low-Cost System Architecture: Successfully integrated a fully functional autonomous UGV platform at a total component hardware cost under $50, establishing an accessible benchmark for research and educational field robotics.

2. Generalized EKF Sensor Fusion Strategy: Developed a kinematically decoupled state estimation pipeline that fuses longitudinal wheel velocity (V_x) with MPU-6050 IMU yaw rate (ω_z). Empirical results demonstrated that EKF fusion reduced short-range translational distance errors from 7.59% down to 0.86% (over 5.8m) and real-world corridor distance errors from 19.54% down to 7.81% (over 52.5m), eliminated heading drift during in-place pivot turns, and improved map feature metric precision to within 2 millimeters (0.28% error on a 0.70m door frame).

3. Dual-Threshold Safety Reflex and Dynamic Covariance Mechanism: Implemented a novel C++ fault-detection controller node (`base_controller_node`) that continuously monitors longitudinal acceleration

(A_x) and asynchronous angular velocity mismatch ($yaw\_mismatch$). The algorithm successfully categorizes severe collisions ($|A_x| > 4.0 m/s^2$), wall scraping ($|A_x| > 2.0 m/s^2$), and wheel slip ($yaw\_mismatch > 0.5 rad/s$), executing emergency braking (<40ms latency) and inflating wheel odometry covariance ($\sigma^2 = 100.0$) to shield the EKF pose graph from corrupt data.

4. Real-World Featureless Environment Resilience: Validated the platform in a long (~52.5m) featureless corridor at VJU My Dinh campus. While standalone LiDAR + Encoder configurations suffered catastrophic topological map collapse during loop closure, the proposed EKF fusion system maintained sharp wall boundaries (~0.10m thickness), cleanly reconstructed glass-partitioned rooms and obstacles, and preserved map integrity across long-range navigation loops.

5.2. Limitations and Future Work

Despite the significant achievements demonstrated in this thesis, several hardware and algorithmic limitations remain, offering promising avenues for future research:

1. 2D Planar Constraint Expansion: The current mapping pipeline operates strictly on a 2D planar assumption (3 Degrees of Freedom: X, Y, Yaw). When traversing severely steep inclines, stairs, or 3D terrain rubble in collapsed caves, 2D SLAM suffers from projection distortion. Future work will integrate 3D LiDAR sensors or stereo depth cameras alongside 9-DOF IMUs (with magnetometer drift correction) to execute full 6-DOF 3D volumetric SLAM (e.g., via RTAB-Map (Labbé and Michaud, 2019) or LIO-SAM).

2. Autonomous Navigation and Path Planning: The current system relies on teleoperated keyboard control (`/cmd_vel`) for environmental exploration. Future development will integrate the ROS2 Navigation Stack (Nav2), incorporating costmap generation, dynamic obstacle avoidance, and

autonomous frontier exploration algorithms to enable fully unassisted robot mapping in unexplored hazard zones.

3. Active Wheel Slip Recovery Control: While the current Bump Reflex mechanism successfully executes emergency braking and EKF covariance inflation upon slip detection, the vehicle relies on manual teleop operator intervention to back away from obstacles. Implementing active adaptive traction control algorithms - such as independent wheel torque vectoring or automated reversing routines - will further enhance autonomous vehicle survivability in extreme mud and subterranean terrains.

4. Long-Range Featureless Corridor Drift: Operating in extended uniform corridors (>50m) still accumulates along-track drift (~3.18m over a 105m loop) due to 2D LiDAR geometric degeneracy, leading to minor localized map overlapping post-loop closure.

REFERENCES

Agarwal, S., Mierle, K. and Ceres Solver Development Team (2022), Ceres Solver: A Large-Scale Non-Linear Least Squares Solver, Google Inc., Mountain View.

Grisetti, G., Stachniss, C. and Burgard, W. (2007), "Improved techniques for grid mapping with Rao-Blackwellized particle filters", IEEE Transactions on Robotics, **23**(1), pp. 34-46.

Hess, W., Kohler, D., Rapp, H. and Andor, D. (2016), "Real-time loop closure in 2D LIDAR SLAM", 2016 IEEE International Conference on Robotics and Automation (ICRA), IEEE, Stockholm, pp. 1271-1278.

InvenSense Inc. (2013), MPU-6050 Product Specification Revision 4.2, InvenSense Motion Processing Unit Data, San Jose.

Karto Robotics (2010), OpenKarto Core Technical Documentation and Scan Matching Algorithms, Karto Robotics Inc., San Jose.

Labbé, M. and Michaud, F. (2019), "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation", Journal of Field Robotics, **36**(2), pp. 416-446.

Macenski, T. and Jambrecic, I. (2021), "SLAM Toolbox: SLAM for the dynamic world", Journal of Open Source Software, **6**(61), pp. 2783.

Moore, T. and Stouch, D. (2014), "A Generalized Extended Kalman Filter Implementation for the Robot Operating System", Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13), Springer, Padua, pp. 335-348.

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R. and Ng, A. Y. (2009), "ROS: an open-source Robot Operating System", ICRA Workshop on Open Source Software, **3**(3.2), pp. 5.

Thrun, S., Burgard, W. and Fox, D. (2005), Probabilistic Robotics, MIT Press, Cambridge.